



UNIVERSITAS INDONESIA

**PENGEMBANGAN *JOINT DOCUMENT LEVEL SENTIMENT-TOPIC MODEL*
BERBASIS *EXTENDED LONG SHORT-TERM MEMORY* UNTUK ANALISIS
SENTIMEN**

SKRIPSI

HAIKAL FIKRI RABANI

2206823713

**FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PROGRAM STUDI SARJANA STATISTIKA
DEPOK
JUNI 2026**



UNIVERSITAS INDONESIA

**PENGEMBANGAN *JOINT DOCUMENT LEVEL SENTIMENT-TOPIC MODEL*
BERBASIS *EXTENDED LONG SHORT-TERM MEMORY* UNTUK ANALISIS
SENTIMEN**

SKRIPSI

**Diajukan sebagai salah satu syarat memperoleh gelar
Sarjana Statistika**

HAIKAL FIKRI RABANI

2206823713

**FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PROGRAM STUDI SARJANA STATISTIKA
DEPOK
JUNI 2026**

HALAMAN PERNYATAAN ORISINALITAS

**Skripsi ini adalah hasil karya saya sendiri,
dan semua sumber baik yang dikutip maupun dirujuk
telah saya nyatakan dengan benar.**

Nama : Haikal Fikri Rabani

NPM : 2206823713

Tanda Tangan

Tanggal : 26 Februari 2026

HALAMAN PENGESAHAN

Skripsi ini diajukan oleh :

Nama : Haikal Fikri Rabani
NPM : 2206823713
Program Studi : Sarjana Statistika
Judul Skripsi : Pengembangan *Joint Document Level Sentiment–Topic Model*
Berbasis *Extended Long Short–Term Memory* untuk Analisis
Sentimen

Telah berhasil dipertahankan di hadapan Dewan Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Statistika pada Program Studi Sarjana Statistika, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Indonesia.

DEWAN PENGUJI

Pembimbing I : Prof. Dr. rer. nat. Hendri Murfi, S.Si., M.Kom. ()

Pembimbing II : Dra. Nora Hariadi, M.Si. ()

Penguji I : Prof. Dr. Drs. Suryadi, M.T. ()

Penguji II : Dr. Dra. Yekti Widyaningsih, M.Si. ()

Ditetapkan di: Depok
Tanggal : 26 Februari 2026

HALAMAN PERNYATAAN PENGGUNAAN KECERDASAN ARTIFISIAL

Saya menyatakan bahwa dalam penyusunan Tugas Akhir ini, saya menggunakan teknologi Kecerdasan Buatan dengan *platform* ChatGPT untuk tujuan penulisan dan saya bertanggung jawab penuh atas keakuratan, orisinalitas, dan integritas seluruh isi Tugas Akhir ini.

Nama : Haikal Fikri Rabani

NPM : 2206823713

Tanda Tangan :

Tanggal : 26 Februari 2026

KATA PENGANTAR

Puji syukur kepada Tuhan Yang Maha Esa atas segala berkat dan rahmat-Nya, sehingga penulis dapat menyelesaikan skripsi ini dengan sebaik-baiknya. Selama masa penulisan skripsi ini, penulis telah mendapat banyak dukungan, doa, bantuan, inspirasi, dan motivasi dari berbagai pihak. Untuk itu, penulis ingin berterima kasih kepada:

1. Bapak Prof. Dr. rer. nat. Hendri Murfi, S.Si., M.Kom. selaku Dosen Pembimbing I yang telah membimbing penulis dengan penuh kesabaran serta memberikan arahan, masukan, dan ilmu yang sangat berharga selama proses penyusunan skripsi ini. Waktu, perhatian, dan dedikasi yang beliau berikan sangat membantu penulis dalam memahami materi serta menyelesaikan penelitian ini dengan baik.
2. Ibu Dra. Nora Hariadi, M.Si. selaku Dosen Pembimbing II yang telah memberikan arahan dan masukan terkait tata cara penulisan skripsi yang sesuai dengan pedoman yang berlaku. Bimbingan beliau sangat membantu penulis dalam menyusun karya ilmiah yang sistematis dan sesuai kaidah akademik.
3. Bapak Prof. Dr. Drs. Suryadi, M.T. selaku Dosen Penguji I yang telah meluangkan waktu untuk mengevaluasi skripsi ini serta memberikan kritik dan saran yang konstruktif. Masukan yang diberikan sangat bermanfaat dalam penyempurnaan penelitian ini.
4. Ibu Dr. Dra. Yekti Widyaningsih, M.Si. selaku Dosen Penguji II yang telah memberikan evaluasi, kritik, dan saran yang membangun sehingga skripsi ini dapat menjadi lebih baik dan layak untuk dipublikasikan.
5. Ibu Kurnia Susvitasari, S.Si., M.Sc., Ph.D. selaku Pembimbing Akademis penulis yang telah memberikan bimbingan, motivasi, serta arahan selama masa perkuliahan. Dukungan beliau sangat berarti dalam pembelajaran akademik penulis.
6. Ibu Prof. Dr. Titin Siswantining, D.E.A. selaku Ketua Pojok Statistik yang telah memberikan ilmu, pengalaman, serta kesempatan berharga dalam pengembangan diri penulis, sehingga membentuk penulis menjadi pribadi yang lebih baik secara akademik maupun personal.
7. Seluruh dosen Departemen Matematika yang telah memberikan ilmu pengetahuan, wawasan, dan pengalaman selama masa perkuliahan, yang menjadi bekal penting bagi penulis dalam menyelesaikan skripsi ini.

8. Keluarga tercinta, yaitu Bapak Eka Adi Susila, Ibu Else Dwi Listiani, dan Adik Miftahul Rizqi Rabani, yang senantiasa memberikan doa, dukungan moral, serta semangat yang tiada henti. Kehadiran dan dukungan mereka menjadi sumber kekuatan utama bagi penulis dalam menyelesaikan studi ini.
9. Ester Oktafia yang telah memberikan dukungan, masukan, serta semangat kepada penulis selama proses penyusunan skripsi ini, sehingga penulis dapat tetap termotivasi dalam menghadapi berbagai tantangan.
10. Teman-teman seperjuangan dari grup "Ujang Kedu" yang telah kebersamai penulis sejak awal perkuliahan hingga tahap akhir studi. Kebersamaan, dukungan, dan pengalaman yang dibagikan menjadi bagian penting dalam perjalanan akademik penulis.
11. Teman-teman dari grup "Citsit Kompstat" yang telah berperan aktif dalam kegiatan belajar bersama, khususnya dalam mempersiapkan ujian, sehingga membantu penulis dalam memahami materi perkuliahan dengan lebih baik.
12. Seluruh *mentee* dari kelompok "Piecewise" yang telah memberikan semangat dan motivasi kepada penulis, serta menjadi pengingat untuk terus berkembang dan menyelesaikan tanggung jawab akademik dengan baik.
13. Tri Hidayatullah dan Edwin Daniel selaku adik asuh di Departemen Matematika yang telah memberikan dukungan serta kebersamaan selama masa perkuliahan.
14. Seluruh pihak yang telah terlibat dan memberikan kontribusi dalam penelitian ini yang tidak dapat disebutkan satu per satu, baik secara langsung maupun tidak langsung.

Penulis menyadari bahwa penyusunan skripsi ini tidak terlepas dari berbagai keterbatasan dan masih memiliki kekurangan, baik dari segi penulisan maupun substansi keilmuan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun sebagai bahan evaluasi untuk perbaikan di masa yang akan datang. Besar harapan penulis agar skripsi ini dapat memberikan manfaat, baik sebagai referensi akademik maupun sebagai kontribusi dalam pengembangan ilmu pengetahuan, khususnya di bidang yang terkait dengan penelitian ini.

Depok, 26 Februari 2026

Haikal Fikri Rabani

**HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR
UNTUK KEPENTINGAN AKADEMIS**

Sebagai sivitas akademik Universitas Indonesia, saya yang bertanda tangan di bawah ini:

Nama : Haikal Fikri Rabani
NPM : 2206823713
Program Studi : Sarjana Statistika
Departemen : Matematika
Fakultas : Matematika dan Ilmu Pengetahuan Alam
Jenis Karya : Skripsi

demikian pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Indonesia **Hak Bebas Royalti Noneksklusif** (*Non-exclusive Royalty Free Right*) atas karya ilmiah saya yang berjudul:

Pengembangan *Joint Document Level Sentiment–Topic Model* Berbasis *Extended Long Short–Term Memory* untuk Analisis Sentimen

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini, Universitas Indonesia berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*data base*), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Depok
Pada tanggal : 26 Februari 2026
Yang menyatakan

(Haikal Fikri Rabani)

ABSTRAK

Nama : Haikal Fikri Rabani

Program Studi : Sarjana Statistika

Judul : Pengembangan *Joint Document Level Sentiment–Topic Model* Berbasis *Extended Long Short–Term Memory* untuk Analisis Sentimen

Peningkatan volume data teks tidak terstruktur dari berbagai platform digital, seperti ulasan produk dan media sosial, mendorong kebutuhan akan metode analisis otomatis untuk mengekstrak opini publik secara efisien. Analisis sentimen menjadi pendekatan utama dalam *Natural Language Processing* (NLP) untuk mengidentifikasi polaritas opini dalam teks. Namun, dalam praktiknya, satu dokumen sering kali memuat beberapa topik atau pembahasan dengan sentimen yang berbeda, sehingga tidak selalu merepresentasikan satu polaritas yang sejenis. Pendekatan konvensional pada tingkat dokumen yang mengasumsikan satu sentimen per dokumen menjadi kurang tepat untuk memodelkan kondisi tersebut. Untuk mengatasi keterbatasan ini, penelitian ini mengusulkan model *Joint Sentiment–Topic* (JST) berbasis *Multi-Task Learning* (MTL) dengan arsitektur *hard parameter sharing*. Model memanfaatkan *Extended Long Short-Term Memory* (xLSTM) sebagai *shared encoder* dan *Deep Embedded K-Means* (DEKM) untuk pendeteksian topik, dengan tahap pra-pelatihan berbasis rekonstruksi *Bag-of-Words* (BoW) untuk meningkatkan kualitas representasi. Evaluasi dilakukan pada dataset IMDB dan Yelp dengan variasi jumlah topik ($K = 30, 50, 80$) dan empat konfigurasi xLSTM (1:0, 0:1, 1:1, 7:1). Hasil menunjukkan bahwa konfigurasi xLSTM[7:1] memberikan kinerja terbaik pada klasifikasi sentimen dengan akurasi 82,33% (IMDB, $K = 30$) dan 90,15% (Yelp, $K = 50$), serta secara konsisten mengungguli LSTM. Pada pendeteksian topik, konfigurasi ini juga menghasilkan koherensi tertinggi pada $K = 30$, dengan nilai 0,4750 (IMDB) dan 0,4782 (Yelp). Selain itu, ditemukan adanya *trade-off* antara performa sentimen dan kualitas topik, di mana peningkatan jumlah topik cenderung menurunkan koherensi semantik. Konfigurasi xLSTM[0:1] menunjukkan performa terendah, yang menegaskan pentingnya dominasi blok mLSTM dalam menghasilkan representasi semantik yang lebih kaya. Secara keseluruhan, model JST berbasis xLSTM terbukti efektif untuk pemodelan sentimen dan topik secara simultan pada tingkat dokumen.

Kata kunci: *Extended Long Short–Term Memory, Deep Embedded K-Means, Joint Sentiment–Topic, Multi–Task Learning.*

ABSTRACT

Name : Haikal Fikri Rabani

Study Program : Undergraduate Study Program of Statistics

Title : Development of a *Joint Sentiment–Topic Model* Based on *Extended Long Short–Term Memory* with a *Multi–Task Learning* Approach for Topic-Level Sentiment Analysis

The increasing volume of unstructured text data from digital platforms, such as product reviews and social media, has driven the need for automated methods to efficiently extract public opinion. Sentiment analysis has become a primary approach in *Natural Language Processing* (NLP) to identify the polarity of opinions in text. However, in practice, a single document often contains multiple topics or discussions with different sentiments, and therefore does not always represent a single, uniform polarity. Conventional document-level sentiment analysis, which assumes one sentiment per document, is thus less suitable for modeling such conditions. To address this limitation, this study proposes a *Joint Sentiment–Topic* (JST) model based on *Multi-Task Learning* (MTL) with a *hard parameter sharing* architecture. The model employs *Extended Long Short-Term Memory* (xLSTM) as a shared encoder and *Deep Embedded K-Means* (DEKM) for topic detection, with a pre-training stage based on *Bag-of-Words* (BoW) reconstruction to improve representation quality. The model is evaluated on the IMDB and Yelp datasets with varying numbers of topics ($K = 30, 50, 80$) and four xLSTM configurations (1:0, 0:1, 1:1, 7:1). The results show that the xLSTM[7:1] configuration achieves the best performance in sentiment classification, with accuracies of 82.33% on IMDB ($K = 30$) and 90.15% on Yelp ($K = 50$), consistently outperforming LSTM. In topic detection, this configuration also yields the highest coherence at $K = 30$, with scores of 0.4750 (IMDB) and 0.4782 (Yelp). Furthermore, a *trade-off* between sentiment performance and topic quality is observed, where increasing the number of topics tends to reduce semantic coherence. The xLSTM[0:1] configuration shows the lowest performance, highlighting the importance of mLSTM-dominant structures in capturing richer semantic representations. Overall, the proposed xLSTM-based JST model is effective for jointly modeling sentiment and topics at the document level.

Keywords: *Extended Long Short–Term Memory, Deep Embedded K-Means, Joint Sentiment–Topic, Multi–Task Learning.*

DAFTAR ISI

HALAMAN PERNYATAAN ORISINALITAS	iii
HALAMAN PENGESAHAN	v
SURAT PERNYATAAN	vii
KATA PENGANTAR	ix
HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI	xi
ABSTRAK	xiii
ABSTRACT	xv
DAFTAR ISI	xvii
DAFTAR TABEL	xxi
DAFTAR GAMBAR	xxiii
DAFTAR LAMPIRAN	xxv
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Tujuan Penelitian	5
1.4 Manfaat Penelitian	5
1.5 Ruang Lingkup Penelitian	6
BAB 2 LANDASAN TEORI	7
2.1 Analisis Sentimen	7
2.2 Pendeteksian Topik	8
2.3 <i>Artificial Neural Network</i>	8

2.3.1	<i>Arsitektur Artificial Neural Network</i>	9
2.3.1.1	<i>Perceptron</i>	10
2.3.1.2	<i>Multi-Layer Perceptron</i>	10
2.3.1.3	Fungsi Aktivasi	11
2.3.1.4	Teknik Normalisasi	15
2.3.1.5	Fungsi <i>Loss</i>	16
2.3.1.6	<i>Backpropagation</i>	18
2.3.1.7	Optimasi Adam with Weight Decay	19
2.4	<i>Recurrent Neural Network</i>	20
2.5	<i>Long Short-Term Memory</i>	22
2.6	Transformer	23
2.6.1	Arsitektur Transformer	24
2.6.2	<i>Word Embedding</i>	24
2.6.3	<i>Encoder</i>	25
2.6.4	Mekanisme <i>Attention</i>	25
2.6.4.1	<i>Scaled Dot-Product Attention</i>	25
2.6.4.2	<i>Multi-Head Attention</i>	26
2.6.5	<i>Skip Connection</i>	26
2.6.6	<i>Decoder</i>	27
2.7	<i>Clustering</i>	28
2.8	<i>Multi-Task Learning</i>	29
2.9	Metrik Evaluasi	30
2.9.1	Metrik Evaluasi Analisis Sentimen	31
2.9.2	Metrik Evaluasi pendeteksian topik	33

BAB 3	METODE <i>JOINT DOCUMENT LEVEL SENTIMENT-TOPIC</i>	
	MODEL BERBASIS <i>EXTENDED LONG SHORT-TERM MEMORY</i>	
	UNTUK ANALISIS SENTIMEN	35
3.1	<i>Extended Long Short-Term Memory</i>	35
3.1.1	<i>Scalar Long Short-Term Memory</i>	36
3.1.1.1	Arsitektur sLSTM	37
3.1.1.2	Blok sLSTM	39
3.1.2	<i>Matrix Long Short-Term Memory</i>	40
3.1.2.1	Arsitektur mLSTM	40
3.1.2.2	Blok mLSTM	42
3.2	<i>Deep Embedded K-Means</i>	43

3.3	Model <i>Joint Sentiment–Topic</i> Berbasis <i>Extended Long Short–Term Memory</i>	45
3.3.1	Pra-pelatihan <i>Encoder Extended Long Short–Term Memory</i>	46
3.3.2	Pembentukan Representasi Laten	46
3.3.3	Jalur Analisis Sentimen	47
3.3.4	Jalur Pendeteksian Topik	47
3.3.5	<i>Joint Loss</i> dan <i>Joint Training</i>	48
BAB 4	HASIL SIMULASI DAN PEMBAHASAN	49
4.1	Spesifikasi Mesin dan Perangkat Lunak	49
4.2	Tahapan Umum Simulasi	49
4.3	Data	51
4.3.1	Pengumpulan Data	51
4.3.2	Prapengolahan Data	52
4.3.3	Representasi Teks	55
4.3.4	Pembagian Data	56
4.4	Implementasi <i>Joint Sentiment–Topic Model</i>	56
4.4.1	Konfigurasi <i>Encoder Extended Long Short–Term Memory</i>	56
4.4.2	Konfigurasi Pra-pelatihan xLSTM	57
4.4.3	Konfigurasi <i>Joint Training</i>	57
4.5	Analisis Kinerja Model <i>Joint Sentiment–Topic</i>	58
4.5.1	Analisis Kinerja Hasil Pra-pelatihan	58
4.5.2	Analisis Kinerja Hasil <i>Joint Training</i>	60
4.5.2.1	Analisis Kinerja Hasil Analisis Sentimen	62
4.5.2.2	Analisis Kinerja Hasil Pendeteksian Topik	63
4.6	Analisis Interpretabilitas Model	63
BAB 5	PENUTUP	65
5.1	Kesimpulan	65
5.2	Saran	65
	DAFTAR PUSTAKA	67

DAFTAR TABEL

Tabel 2.1	<i>Ilustrasi Confusion Matrix</i>	31
Tabel 4.1	Spesifikasi mesin dan perangkat lunak	49
Tabel 4.2	Cuplikan data ulasan IMDB (Maas et al., 2011).	51
Tabel 4.3	Cuplikan data ulasan Yelp (Zhang et al., 2015).	52
Tabel 4.4	Contoh hasil prapengolahan pada dataset IMDB dan Yelp	54
Tabel 4.5	Contoh proses tokenisasi	55
Tabel 4.6	Contoh proses <i>embedding</i>	56
Tabel 4.7	Konfigurasi utama <i>encoder</i> xLSTM	57
Tabel 4.8	Konfigurasi utama pra-pelatihan xLSTM	57
Tabel 4.9	Konfigurasi utama <i>joint training</i> model JST	58
Tabel 4.10	Perbandingan <i>train loss</i> dan <i>validation loss</i> pra-pelatihan	59
Tabel 4.11	Hasil evaluasi model pada dataset IMDB dengan $K = 30$	60
Tabel 4.12	Hasil evaluasi model pada dataset Yelp dengan $K = 30$	60
Tabel 4.13	Hasil evaluasi model pada dataset IMDB dengan $K = 50$	61
Tabel 4.14	Hasil evaluasi model pada dataset Yelp dengan $K = 50$	61
Tabel 4.15	Hasil evaluasi model pada dataset IMDB dengan $K = 80$	61
Tabel 4.16	Hasil evaluasi model pada dataset Yelp dengan $K = 80$	62

DAFTAR GAMBAR

Gambar 2.1	Struktur neuron biologis (Aggarwal et al., 2018).	8
Gambar 2.2	Struktur MLP (Aggarwal et al., 2018).	11
Gambar 2.3	Grafik fungsi sigmoid.	12
Gambar 2.4	Grafik fungsi tanh.	13
Gambar 2.5	Grafik fungsi eksponensial.	13
Gambar 2.6	Grafik fungsi GeLU.	14
Gambar 2.7	Grafik fungsi Swish.	14
Gambar 2.8	Ilustrasi graf komputasi MLP (Aggarwal et al., 2018), telah diolah kembali.	18
Gambar 2.9	Arsitektur RNN (Mienye et al., 2024).	21
Gambar 2.10	Arsitektur dasar LSTM (Mienye et al., 2024).	22
Gambar 2.11	Arsitektur dasar Transformer (Vaswani et al., 2023), telah diolah kembali.	24
Gambar 2.12	Ilustrasi proses pembentukan <i>query</i> , <i>key</i> , dan <i>value</i> dari <i>input embedding</i> (Pradipta Hardinatha et al., 2025), telah diolah kembali.	26
Gambar 2.13	Ilustrasi <i>skip connection</i> (Xu et al., 2025).	27
Gambar 2.14	Ilustrasi pendekatan <i>hard parameter sharing</i> (Ruder, 2017), telah diolah kembali.	30
Gambar 3.1	Arsitektur model JST berbasis xLSTM	35
Gambar 3.2	Arsitektur xLSTM (Beck et al., 2024).	36
Gambar 3.3	Arsitektur sLSTM (Alharthi & Mahmood, 2024), telah diolah kembali.	37
Gambar 3.4	Arsitektur blok sLSTM (Beck et al., 2024).	39
Gambar 3.5	Arsitektur mLSTM (Alharthi & Mahmood, 2024), telah diolah kembali.	40
Gambar 3.6	Arsitektur Blok mLSTM (Beck et al., 2024).	43
Gambar 3.7	Alur lengkap jalur analisis sentimen	47
Gambar 3.8	Alur lengkap jalur pendeteksian topik	48

Gambar 4.1	Diagram alur penelitian.	50
Gambar 4.2	Perbandingan <i>train loss</i> pra-pelatihan seluruh <i>encoder</i>	59
Gambar 4.3	Perbandingan <i>validation loss</i> pra-pelatihan seluruh <i>encoder</i>	60

DAFTAR LAMPIRAN

<i>Code</i> Pra-pelatihan xLSTM	73
<i>Code Joint Sentiment–Topic</i> berbasis xLSTM	75

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Pada era digital, volume data teks tidak terstruktur dari platform daring seperti media sosial, ulasan produk, dan portal berita mengalami peningkatan yang masif. Setiap hari, jutaan pengguna menghasilkan konten yang mencerminkan opini, emosi, dan persepsi mereka terhadap berbagai topik, menciptakan ledakan informasi yang kaya akan wawasan (Andreotta et al., 2019). Kondisi ini membuka peluang besar untuk memahami opini publik dan tren sosial, namun sekaligus menghadirkan tantangan baru karena proses analisis manual tidak efisien. Sebagaimana dikemukakan oleh (Nikhil, 2015), sebagian besar data di internet bersifat tidak terstruktur dan memerlukan pendekatan komputasional seperti *text mining* dan *Natural Language Processing* (NLP) agar dapat dimanfaatkan secara optimal.

Salah satu cabang utama NLP yang berkembang pesat sampai saat ini adalah analisis sentimen, yaitu proses mengidentifikasi opini dan emosi yang terkandung dalam teks. Evolusi metode analisis sentimen berawal dari pendekatan berbasis aturan dan leksikon, di mana daftar kata dengan polaritas positif dan negatif digunakan untuk menentukan orientasi sentimen teks (Choi & Cardie, 2009). Seiring meningkatnya kompleksitas data, analisis sentimen berkembang ke level dokumen yang mengklasifikasikan keseluruhan teks sebagai satu kesatuan opini (Behdenna et al., 2018). Meskipun efektif untuk teks sederhana, metode ini gagal menangkap variasi opini terhadap berbagai aspek atau topik dalam satu dokumen (Zhang et al., 2022).

Dalam perkembangannya, analisis sentimen dapat dilakukan pada berbagai tingkat analisis, salah satunya adalah tingkat dokumen (*document-level sentiment analysis*), di mana setiap dokumen diasumsikan memiliki satu polaritas sentimen, seperti positif, negatif, atau netral (Pang & Lee, 2008). Pendekatan ini memiliki keunggulan berupa struktur yang sederhana, efisiensi komputasi yang tinggi, serta kebutuhan anotasi yang relatif minimal, sehingga banyak diterapkan dalam berbagai bidang (Zhang et al., 2018).

Meskipun demikian, pendekatan pada tingkat dokumen masih memiliki keterbatasan dalam menangkap konteks tema dari opini yang dianalisis. Dalam praktiknya, sebuah dokumen seperti ulasan atau komentar umumnya berfokus pada satu topik utama, namun ketika dikumpulkan dalam suatu korpus, terdapat banyak dokumen yang membahas topik serupa dengan variasi sentimen yang berbeda. Sebagai contoh, dalam suatu kumpulan ulasan terhadap sebuah produk, beberapa dokumen dapat memberikan penilaian positif,

sementara dokumen lain memberikan penilaian negatif terhadap produk yang sama. Hal ini menunjukkan bahwa meskipun dokumen-dokumen tersebut berada dalam topik yang serupa, distribusi sentimen yang dihasilkan dapat bervariasi (Pang & Lee, 2008). Dengan kata lain, informasi sentimen akan menjadi lebih bermakna apabila dianalisis bersama dengan konteks topik yang mendasari dokumen tersebut.

Untuk mengatasi keterbatasan tersebut, berkembang pendekatan yang menggabungkan analisis sentimen dengan pendeteksian topik pada tingkat dokumen. Pendekatan ini bertujuan untuk mengelompokkan dokumen berdasarkan topik yang serupa, kemudian menganalisis distribusi sentimen pada setiap kelompok topik tersebut. Secara umum, terdapat dua paradigma utama dalam pendekatan ini, yaitu pendekatan *two-stage pipeline* dan *joint modeling*. Pada pendekatan *two-stage pipeline*, proses pendeteksian topik dan klasifikasi sentimen dilakukan secara terpisah. Umumnya, topik diidentifikasi terlebih dahulu menggunakan metode seperti *Latent Dirichlet Allocation* (LDA) (Blei, 2012), kemudian hasilnya digunakan sebagai fitur dalam proses klasifikasi sentimen menggunakan algoritma *machine learning* seperti *Support Vector Machine* (SVM). Meskipun pendekatan ini relatif sederhana dan mudah diimplementasikan, pemisahan kedua tahap tersebut menyebabkan model rentan terhadap propagasi kesalahan serta belum mampu menangkap keterkaitan intrinsik antara struktur topik dan distribusi sentimen secara optimal (Sengupta et al., 2021).

Sebagai alternatif dari pendekatan tersebut, berkembang pendekatan *joint modeling* yang mengintegrasikan pemodelan topik dan analisis sentimen dalam satu kerangka terpadu. Salah satu model yang umum digunakan dalam pendekatan ini adalah *Joint Sentiment-Topic* (JST) yang merupakan pengembangan dari *Latent Dirichlet Allocation* (LDA) dengan menambahkan variabel sentimen ke dalam proses generatif (Lin, 2009). Model JST memungkinkan setiap kata dalam dokumen memiliki distribusi bersama antara topik dan sentimen, sehingga hubungan antara kedua dimensi tersebut dapat dipelajari secara simultan dalam satu model.

Perkembangan berikutnya berfokus pada peningkatan kemampuan JST dalam menangani data teks yang memuat beberapa topik serta variasi sentimen dalam satu dokumen dan membutuhkan pemahaman konteks antar kata. *Topic Sentiment LDA* (TSLDA) (Nguyen & Shirai, 2015) memperluas kerangka probabilistik JST agar lebih fleksibel dalam merepresentasikan hubungan antara topik dan sentimen, sementara *Topic Sentiment model with Word Embedding* (TSWE) (Fu et al., 2016) mengintegrasikan *word embedding* untuk menangkap kesamaan semantik pada teks pendek. Pengembangan lebih lanjut dilakukan melalui *Embedding-enhanced JST* (ELJST) (Sengupta et al., 2021) yang mengintegrasikan *word embedding* berbasis *skip-gram* dengan regularisasi

Markov Random Field untuk memperkaya representasi semantik dan meningkatkan akurasi identifikasi topik serta sentimen pada teks pendek. Integrasi representasi berbasis *embedding* ini menunjukkan bahwa penggunaan informasi semantik dapat meningkatkan kualitas pemodelan topik dan sentimen, serta membantu model dalam memahami hubungan antar kata secara lebih baik.

Perkembangan metode NLP berbasis *deep learning* telah meningkatkan kemampuan dalam pengolahan data teks yang kompleks. Salah satu arsitektur penting dalam kemajuan ini adalah *Long Short-Term Memory* (LSTM) yang diperkenalkan oleh Sepp Hochreiter dan Jürgen Schmidhuber pada tahun 1997 (Vennerød et al., 2021). LSTM merupakan pengembangan dari *Recurrent Neural Network* (RNN) yang dirancang untuk mengatasi masalah *vanishing gradient* melalui mekanisme *input*, *forget*, dan *output gate*. Struktur ini memungkinkan model mempertahankan informasi penting dalam urutan kata dan mengabaikan konteks yang tidak relevan, sehingga efektif dalam menangkap informasi sentimen dalam teks. Arsitektur LSTM kemudian menjadi dasar bagi berbagai pengembangan model *neural* modern, salah satunya *Extended Long Short-Term Memory* (xLSTM) yang dikembangkan untuk meningkatkan kapasitas pemodelan konteks dan efisiensi pelatihan melalui mekanisme *exponential gating* dan *matrix memory* (Beck et al., 2024; Alharthi & Mahmood, 2024). Pengembangan ini menunjukkan bahwa arsitektur berbasis memori masih relevan dan terus berevolusi dalam menghadapi kompleksitas representasi bahasa alami.

Salah satu penelitian terbaru yang berupaya menggabungkan analisis sentimen dan pendeteksian topik secara bersamaan adalah penelitian yang dilakukan oleh (Gui et al., 2022) melalui pendekatan *Multi-Task Learning* (MTL). Model ini memanfaatkan dua komponen utama, yaitu klasifikasi sentimen berbasis *Gated Recurrent Unit* (GRU) sebagai varian LSTM dengan struktur *gate* yang lebih sederhana, dan pendeteksian topik berbasis *Variational Autoencoder* (VAE) untuk menghasilkan representasi laten dari teks. Kedua modul tersebut dihubungkan melalui mekanisme *mutual learning* yang menyelaraskan distribusi topik dari model VAE dengan vektor *attention* kata dari model GRU, sehingga memungkinkan pertukaran informasi semantik antar-tugas. Representasi laten hasil VAE kemudian dikelompokkan menggunakan algoritma *K-Means* untuk membentuk topik-topik laten secara eksplisit. Pendekatan ini berhasil meningkatkan akurasi klasifikasi dan koherensi topik, namun karena kedua tugas dilatih secara terpisah, interaksi antar-tugas belum sepenuhnya terintegrasi dalam satu proses pelatihan bersama.

Berdasarkan perkembangan tersebut, penelitian ini mengusulkan pendekatan *multi-task learning* yang mengintegrasikan analisis sentimen dan pendeteksian topik dalam satu proses pelatihan. Pendekatan ini dirancang dengan memanfaatkan arsitektur

xLSTM sebagai representasi bersama untuk mempelajari representasi laten teks, serta *Deep Embedded K-Means* (DEKM) sebagai komponen pendeteksian topik dalam ruang *embedding*. xLSTM digunakan untuk menangkap konteks emosional dan dependensi jangka panjang dalam teks melalui mekanisme *exponential gating* dan *matrix memory* (Beck et al., 2024; Alharthi & Mahmood, 2024), sedangkan DEKM memungkinkan proses pelatihan representasi dan *clustering* dilakukan secara terintegrasi (Guo et al., 2021). Berbeda dengan penelitian (Gui et al., 2022) yang menggabungkan dua modul terpisah melalui mekanisme *mutual learning*, pendekatan dalam penelitian ini mengadopsi skema *hard parameter sharing* sehingga kedua tugas dipelajari secara langsung dalam satu arsitektur terpadu. Dengan demikian, interaksi antara dimensi sentimen dan topik dapat dimodelkan secara lebih konsisten dalam satu representasi bersama.

Untuk menilai efektivitas pendekatan yang diusulkan, dilakukan evaluasi komparatif terhadap metode yang umum digunakan dalam analisis sentimen. Pada tugas analisis sentimen, arsitektur LSTM sering dijadikan acuan karena kemampuannya dalam menangkap dependensi sekuensial dalam teks. Oleh karena itu, kinerja komponen analisis sentimen berbasis xLSTM dalam kerangka JST yang diusulkan dibandingkan dengan representasi model LSTM untuk mengevaluasi peningkatan kualitas representasi konteks dan stabilitas pelatihan.

Sementara itu, pada tugas pendeteksian topik, evaluasi difokuskan pada kualitas topik yang dihasilkan oleh model melalui metrik *topic coherence*. Evaluasi ini bertujuan untuk menilai sejauh mana integrasi DEKM dalam kerangka pelatihan terpadu mampu menghasilkan struktur topik yang koheren dan mudah diinterpretasikan. Dengan demikian, evaluasi yang dilakukan diharapkan dapat memberikan gambaran yang lebih objektif mengenai kinerja masing-masing komponen serta efektivitas pendekatan yang diusulkan secara keseluruhan.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, penelitian ini merumuskan permasalahan sebagai berikut:

1. Bagaimana mengembangkan model *Joint Sentiment–Topic* berbasis xLSTM dengan pendekatan *multi-task learning* yang mengintegrasikan analisis sentimen dan pendeteksian topik dalam satu proses pelatihan?
2. Bagaimana perbandingan kinerja analisis sentimen model *Joint Sentiment–Topic*

berbasis xLSTM dan LSTM pada dataset IMDB dan Yelp?

3. Bagaimana perbandingan kualitas pendeteksian topik model *Joint Sentiment–Topic* berbasis xLSTM dan LSTM pada dataset IMDB dan Yelp?

1.3 Tujuan Penelitian

Berdasarkan masalah penelitian yang telah dirumuskan, maka tujuan dari penelitian ini adalah sebagai berikut:

1. Mengembangkan model *Joint Sentiment–Topic* berbasis xLSTM dengan pendekatan *multi-task learning* yang mengintegrasikan analisis sentimen dan pendeteksian topik dalam satu proses pelatihan.
2. Membandingkan kinerja analisis sentimen model *Joint Sentiment–Topic* berbasis xLSTM dan LSTM pada dataset IMDB dan Yelp.
3. Membandingkan kualitas pendeteksian topik model *Joint Sentiment–Topic* berbasis xLSTM dan LSTM pada dataset IMDB dan Yelp.

1.4 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat baik secara teoretis maupun praktis, yang diuraikan sebagai berikut:

1. Manfaat Teoretis

Penelitian ini diharapkan dapat memberikan kontribusi dalam pengembangan kajian NLP, khususnya pada integrasi analisis sentimen dan pemodelan topik dalam satu kerangka. Selain itu, penelitian ini juga memperkaya literatur terkait pemanfaatan arsitektur xLSTM sebagai representasi bersama dalam pelatihan teks yang digunakan secara simultan untuk beberapa tugas. Hasil penelitian ini diharapkan dapat menjadi referensi bagi penelitian selanjutnya dalam mengembangkan model terpadu yang lebih efektif untuk analisis teks.

2. Manfaat Praktis

Penelitian ini diharapkan dapat memberikan solusi dalam menganalisis data ulasan teks secara lebih komprehensif dengan menggabungkan informasi sentimen dan topik dalam satu proses analisis. Model yang diusulkan dapat dimanfaatkan untuk berbagai kebutuhan, seperti analisis opini pelanggan, pemantauan kepuasan

pengguna, serta pengambilan keputusan berbasis data pada platform digital. Selain itu, pendekatan yang digunakan juga berpotensi meningkatkan efisiensi pemrosesan dibandingkan dengan pendekatan terpisah.

1.5 Ruang Lingkup Penelitian

Batasan dalam penelitian ini ditetapkan sebagai berikut:

1. Penelitian ini menggunakan analisis sentimen biner yang terdiri atas dua kelas, yaitu positif dan negatif.
2. Data yang digunakan merupakan *dataset* IMDB dan Yelp yang berupa kumpulan teks ulasan, yang diperoleh melalui platform Hugging Face, masing-masing pada *dataset* IMDB dan *dataset* Yelp Polarity.
3. Jumlah data ulasan yang digunakan dalam penelitian ini dibatasi sebanyak 30.000 ulasan untuk masing-masing *dataset*, baik IMDB maupun Yelp.

BAB 2

LANDASAN TEORI

Bab ini memaparkan landasan teori yang mendasari pengembangan model *Joint Sentiment–Topic* berbasis xLSTM. Pembahasan mencakup konsep analisis sentimen, pendeteksian topik, arsitektur *deep learning*, serta metode pelatihan dan evaluasi model yang relevan dengan tujuan penelitian.

2.1 Analisis Sentimen

Penjelasan pada subbab ini mengacu pada penelitian (Liu, 2012), kecuali jika disebutkan dari sumber lain. Analisis sentimen atau *opinion mining* merupakan bidang studi dalam NLP yang bertujuan untuk mengekstraksi dan mengevaluasi opini dari data teks. Opini yang dianalisis dapat berupa sentimen, penilaian, atau emosi yang diekspresikan oleh individu terhadap suatu entitas tertentu. Entitas tersebut dapat berupa produk, layanan, organisasi, maupun isu tertentu. Analisis sentimen berperan penting dalam memahami opini dan persepsi yang terkandung dalam data teks tidak terstruktur. Analisis sentimen dapat diklasifikasikan menjadi tiga tingkatan, yaitu:

1. Analisis Sentimen Tingkat Dokumen

Analisis sentimen tingkat dokumen bertujuan untuk menentukan kecenderungan sentimen secara keseluruhan dari sebuah dokumen opini dengan mengelompokkannya ke dalam kategori sentimen positif, netral, atau negatif. Pendekatan ini didasarkan pada asumsi bahwa satu dokumen merepresentasikan satu opini utama terhadap satu entitas tertentu. Analisis sentimen tingkat dokumen lebih cocok digunakan saat keseluruhan isi dokumen mencerminkan pandangan seorang penulis terhadap satu objek secara konsisten.

2. Analisis Sentimen Tingkat Kalimat

Analisis sentimen tingkat kalimat bertujuan untuk mengidentifikasi dan mengklasifikasi sentimen pada setiap kalimat secara terpisah. Proses ini melibatkan dua tahap, yaitu klasifikasi subjektivitas dan klasifikasi sentimen. Klasifikasi subjektivitas digunakan untuk membedakan kalimat subjektif dan kalimat objektif. Kalimat subjektif dianalisis untuk menentukan polaritas sentimen, seperti positif atau negatif, sedangkan kalimat objektif umumnya dikategorikan sebagai netral.

3. Analisis Sentimen Tingkat Aspek

Analisis sentimen tingkat dokumen dan tingkat kalimat lebih sesuai digunakan

ketika teks hanya memuat satu entitas utama. Pada teks yang memuat beberapa aspek dari suatu entitas, analisis sentimen tingkat aspek digunakan untuk mengidentifikasi aspek-aspek tersebut beserta kecenderungan sentimennya, seperti positif, netral, atau negatif.

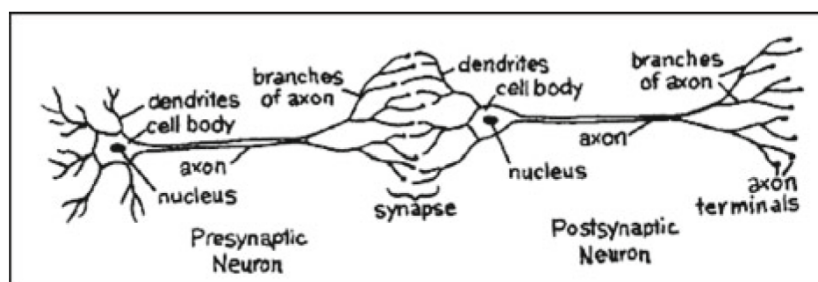
Pada penelitian ini, tingkat analisis sentimen yang digunakan adalah analisis sentimen tingkat dokumen. Setiap ulasan pada data yang digunakan dalam penelitian ini dianggap sebagai satu kesatuan teks yang mencerminkan polaritas sentimen secara keseluruhan, baik positif maupun negatif.

2.2 Pendeteksian Topik

Penjelasan pada subbab ini mengacu pada penelitian (Zafarani-Moattar et al., 2024), kecuali jika disebutkan berasal dari sumber lain. Pendeteksian topik (*topic detection*) merupakan proses untuk mengidentifikasi dan mengelompokkan dokumen berdasarkan kesamaan tema atau topik yang dibahas. Pada data teks, proses ini bertujuan untuk menemukan pola kemunculan kata yang mencerminkan suatu topik tertentu. Pendeteksian topik dipengaruhi oleh karakteristik bahasa serta hubungan semantik antar kata dan dokumen.

Metode pendeteksian topik dapat dikategorikan ke dalam pendekatan *frequent pattern mining* dan *clustering-based*. Pendekatan *frequent pattern mining* berfokus pada identifikasi kata atau kombinasi kata yang sering muncul bersama dalam dokumen sebagai representasi dari suatu topik. Sementara itu, pada pendekatan *clustering-based*, dokumen direpresentasikan dalam bentuk vektor dan kemudian dikelompokkan berdasarkan ukuran kemiripan atau jarak antardokumen. Setiap *cluster* yang terbentuk merepresentasikan suatu topik tertentu.

2.3 Artificial Neural Network



Gambar 2.1: Struktur neuron biologis (Aggarwal et al., 2018).

Artificial Neural Network (ANN) atau jaringan saraf tiruan merupakan model komputasi dalam *machine learning* yang terinspirasi dari sistem saraf biologis, khususnya otak manusia (Aggarwal et al., 2018). Sistem saraf biologis terdiri dari neuron yang berfungsi menerima, mengolah, dan meneruskan informasi melalui komponen utama seperti dendrit, *cell body*, akson, dan sinapsis. Berdasarkan Gambar 2.1, sinyal diterima melalui dendrit dan diproses pada *cell body*. Selanjutnya, sinyal diteruskan melalui akson ke neuron lain melalui sinapsis. Hubungan antarneuron tersebut memiliki tingkat pengaruh yang dapat berubah, sehingga memungkinkan terjadinya proses pembelajaran.

Neuron dalam ANN digambarkan sebagai unit komputasi yang menerima *input* numerik dan masing-masing dikalikan dengan bobot untuk menunjukkan kekuatan hubungan. Jumlah tertimbang ini kemudian diproses oleh fungsi aktivasi untuk menghasilkan *output*. Dengan ini, ANN menggambarkan cara sistem biologis memproses dan meneruskan informasi dalam bentuk numerik (Aggarwal et al., 2018).

ANN dapat mempelajari pola dan fitur penting dari data *input*. Proses ini dikenal sebagai *representation learning*, yaitu pembelajaran representasi data agar informasi penting di dalam data dapat digunakan oleh model untuk menyelesaikan suatu tugas (Bengio et al., 2012). Hasil dari proses tersebut disebut representasi laten, yaitu vektor numerik yang dibentuk oleh lapisan tersembunyi (*hidden layer*) untuk merangkum fitur penting dari data. Kumpulan representasi laten ini membentuk ruang laten, yaitu ruang vektor di mana setiap data dipetakan berdasarkan fitur yang dipelajari oleh model.

2.3.1 Arsitektur *Artificial Neural Network*

Secara umum, arsitektur ANN terdiri atas beberapa lapisan atau *layer* neuron yang saling terhubung dan bekerja secara berurutan dalam memproses informasi (Qamar & Zardari, 2023). *Input layer* berfungsi sebagai titik masuk data ke dalam neuron, di mana *input* berupa fitur diterima dan kemudian diteruskan ke *layer* berikutnya untuk diproses lebih lanjut. Selanjutnya, *layer* yang berada di antara *input layer* dan *output layer*, yang dikenal sebagai *hidden layer*, mengekstraksi pola dan karakteristik penting dari data melalui kombinasi linier dengan bobot dan fungsi nonlinier. Sementara itu, *output layer* menghasilkan *output* akhir neuron sesuai dengan tujuan pemodelan, seperti nilai kontinu pada tugas regresi atau label kelas pada tugas klasifikasi. Perbedaan jumlah *layer*, neuron, serta pola konektivitas antar-neuron menghasilkan berbagai variasi arsitektur ANN, mulai dari jaringan *feedforward* yang sederhana hingga arsitektur yang lebih kompleks (Da Silva et al., 2016).

2.3.1.1 *Perceptron*

Perceptron merupakan model dasar dari ANN yang terdiri atas satu *input layer* yang terhubung langsung ke satu neuron *output* tanpa *hidden layer*. Setiap *input* direpresentasikan sebagai sinyal numerik yang dikalikan dengan bobot tertentu. Hasilnya kemudian dijumlahkan dan diproses menggunakan fungsi aktivasi untuk menghasilkan *output* (Aggarwal et al., 2018).

Arsitektur *perceptron* umumnya dilengkapi dengan komponen bias b yang berfungsi menggeser ambang aktivasi neuron sehingga pengambilan keputusan tidak harus selalu melalui titik asal. Dengan adanya bias, model memiliki fleksibilitas yang lebih besar dalam membentuk batas keputusan yang sesuai dengan distribusi data. Secara matematis, *output* perceptron dirumuskan sebagai

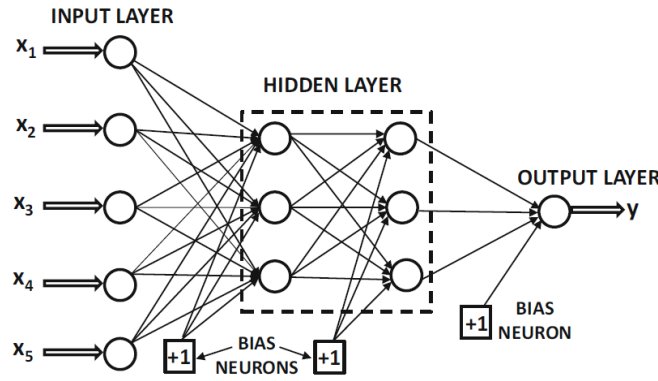
$$y = f \left(\sum_{i=1}^n w_i x_i + b \right), \quad (2.1)$$

dengan x_i menyatakan *input* ke- i , w_i adalah bobot yang merepresentasikan kontribusi relatif setiap *input* terhadap *output* neuron, b adalah bias, dan $f(\cdot)$ adalah fungsi aktivasi yang menentukan respons akhir neuron terhadap sinyal yang diterima. Ketika $b = 0$, persamaan ini menjadi bentuk *perceptron* tanpa bias, sedangkan nilai $b \neq 0$ memungkinkan model untuk lebih adaptif terhadap variasi distribusi data.

2.3.1.2 *Multi-Layer Perceptron*

Multi-Layer Perceptron (MLP) merupakan pengembangan dari model *perceptron* sederhana dengan menambahkan satu atau lebih *hidden layer* di antara *layer input* dan *layer output* (Aggarwal et al., 2018). Keberadaan *hidden layer* memungkinkan jaringan untuk memodelkan hubungan nonlinier yang tidak dapat dijelaskan oleh *perceptron*. Pada arsitektur MLP, setiap neuron pada suatu *layer* terhubung secara penuh dengan neuron pada *layer* berikutnya, sehingga proses pemrosesan informasi berlangsung secara berurutan dari *input layer* hingga *output layer*. Oleh karena itu, MLP diklasifikasikan sebagai *feed-forward neural network*, di mana aliran data hanya terjadi dalam satu arah tanpa koneksi umpan balik.

Struktur umum MLP ditunjukkan pada Gambar 2.2. Pada gambar tersebut, *layer input* menerima vektor *input* yang kemudian diproses secara bertahap oleh *hidden layer* melalui kombinasi linier berbobot dan fungsi aktivasi. Selanjutnya, *output* dari *hidden layer* terakhir diteruskan ke *output layer* untuk menghasilkan *output* akhir jaringan.



Gambar 2.2: Struktur MLP (Aggarwal et al., 2018).

Secara matematis, alur pemrosesan data pada MLP dapat dinyatakan secara berurutan antar *layer*. Misalkan vektor aktivasi pada *layer* ke- r dinyatakan sebagai $\mathbf{h}_r$, maka transformasi dari *input layer* ke *hidden layer* pertama dapat dituliskan sebagai

$$\mathbf{h}_1 = f(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \quad (2.2)$$

dengan $\mathbf{x}$ merupakan vektor *input*, $\mathbf{W}_1$ adalah matriks bobot yang menghubungkan *input layer* dengan *hidden layer* pertama, dan $\mathbf{b}_1$ menyatakan vektor bias. Untuk *hidden layer* selanjutnya, transformasi antar *layer* dapat dinyatakan sebagai

$$\mathbf{h}_{r+1} = f(\mathbf{W}_{r+1} \mathbf{h}_r + \mathbf{b}_{r+1}), \quad r = 1, 2, \dots, l-1, \quad (2.3)$$

dengan l menyatakan jumlah *hidden layer*. Selanjutnya, *output* jaringan diperoleh melalui *output layer* dengan persamaan

$$\mathbf{y} = f(\mathbf{W}_{l+1} \mathbf{h}_l + \mathbf{b}_{l+1}), \quad (2.4)$$

dengan $\mathbf{y}$ merupakan vektor *output* jaringan. Fungsi $f(\cdot)$ pada setiap *layer* merepresentasikan fungsi aktivasi yang digunakan untuk memperkenalkan sifat nonlinier ke dalam model.

2.3.1.3 Fungsi Aktivasi

Fungsi aktivasi digunakan dalam *neural network* untuk memperkenalkan sifat nonlinier ke dalam model. Setelah *input* dikombinasikan dengan bobot dan bias, fungsi aktivasi memetakan hasil tersebut menjadi *output* neuron. Sifat nonlinieritas ini memungkinkan *neural network* mempelajari pola yang lebih kompleks dibandingkan dengan model linier sederhana (Dubey et al., 2022). Secara umum, *neural network*

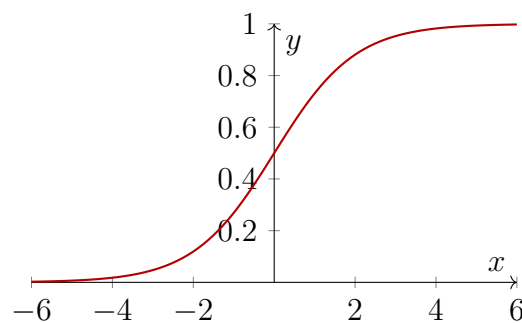
memiliki berbagai jenis fungsi aktivasi yang berbeda yang dapat digunakan. Berikut adalah penjelasan tentang beberapa fungsi aktivasi yang digunakan dalam penelitian:

1. Fungsi Sigmoid

Fungsi sigmoid memetakan *input* ke dalam rentang $[0, 1]$ dan didefinisikan pada Persamaan (2.5):

$$f(x) = \frac{1}{1 + e^{-x}}. \quad (2.5)$$

Berdasarkan sifat pemetaan tersebut, *output* fungsi sigmoid dapat diinterpretasikan sebagai probabilitas, sehingga fungsi ini banyak digunakan pada *layer output* untuk permasalahan klasifikasi biner. Nilai *output* fungsi sigmoid berubah paling signifikan pada kisaran nilai *input* di sekitar nol, serta secara bertahap mendekati nilai 0 dan 1 ketika *input* bernilai sangat kecil atau sangat besar (Nieradzik et al., 2021). Representasi grafis dari fungsi sigmoid ditunjukkan pada Gambar 2.3.



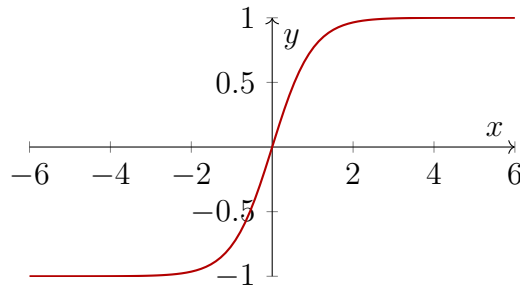
Gambar 2.3: Grafik fungsi sigmoid.

2. Fungsi Hyperbolic Tangent

Fungsi *Hyperbolic tangent* ($\tanh$) merupakan fungsi nonlinier yang memetakan nilai *input* ke dalam rentang $(-1, 1)$, sebagaimana dirumuskan pada Persamaan (2.6):

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}. \quad (2.6)$$

Karena sifatnya yang simetris terhadap nol, fungsi $\tanh$ menghasilkan *output* yang terpusat di sekitar nol sehingga dapat membantu mempercepat konvergensi selama proses pelatihan *neural network* (Dubey et al., 2022). Perubahan nilai *output* paling signifikan terjadi pada kisaran *input* di sekitar nol, sedangkan pada nilai *input* yang ekstrem, fungsi ini cenderung mendekati batas -1 dan 1 . Grafik fungsi $\tanh$ ditunjukkan pada Gambar 2.4.



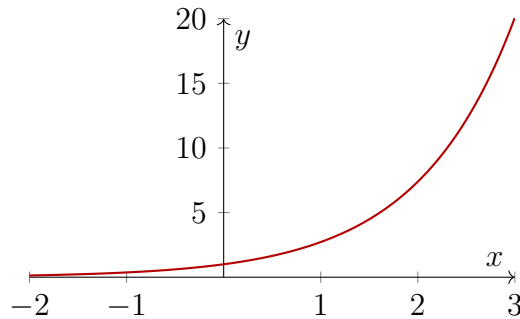
Gambar 2.4: Grafik fungsi tanh.

3. Fungsi Eksponensial

Fungsi eksponensial merupakan fungsi nonlinier yang memetakan setiap nilai *input* ke *output* bernilai positif dan bersifat monoton meningkat, yang ditunjukkan pada Persamaan (2.7):

$$f(x) = e^x. \quad (2.7)$$

Fungsi eksponensial memiliki domain bilangan riil dengan *range* $(0, \infty)$, sehingga perbedaan kecil pada nilai *input* dapat menghasilkan perbedaan *output* yang jauh lebih besar (Ho et al., 2012). Grafik fungsi eksponensial ditunjukkan pada Gambar 2.5.



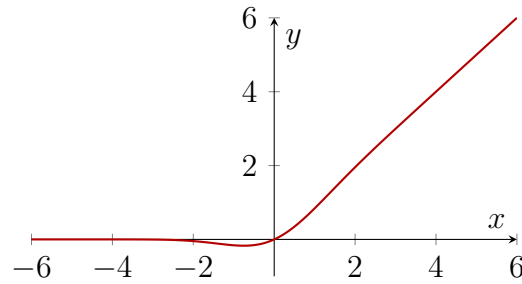
Gambar 2.5: Grafik fungsi eksponensial.

4. Gaussian Error Linear Unit

Fungsi *Gaussian Error Linear Unit* (GeLU) merupakan fungsi aktivasi nonlinier yang menggabungkan nilai *input* dengan fungsi distribusi kumulatif Gaussian, sehingga aktivasi neuron ditentukan secara probabilistik berdasarkan besar kecilnya nilai *input* (Hendrycks & Gimpel, 2023). Secara matematis, fungsi GeLU didefinisikan pada Persamaan (2.8):

$$f(x) = x \cdot P(X \leq x) = x \cdot \Phi(x) = x \cdot \frac{1}{2} \left[1 + \operatorname{erf} \left(\frac{x}{\sqrt{2}} \right) \right], \quad (2.8)$$

dengan $\Phi(x)$ menyatakan fungsi distribusi kumulatif Gaussian standar dan $\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$ merupakan fungsi *error* Gaussian. Fungsi GeLU memberikan pembobotan yang bersifat halus dan kontinu, sehingga umum digunakan pada arsitektur *neural network* untuk memperkenalkan nonlinieritas yang lebih beragam (Dubey et al., 2022). Grafik fungsi GeLU ditunjukkan pada Gambar 2.6.



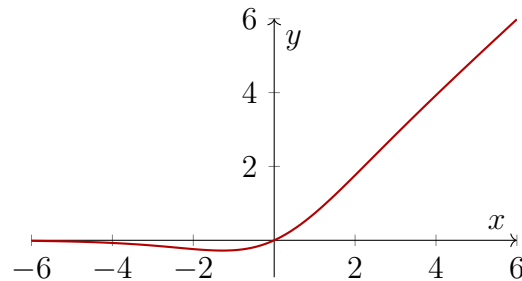
Gambar 2.6: Grafik fungsi GeLU.

5. Swish

Fungsi Swish merupakan fungsi aktivasi nonlinier yang memanfaatkan fungsi sigmoid untuk mengatur besarnya aktivasi dari nilai *input*. Secara matematis, fungsi Swish didefinisikan pada Persamaan (2.9):

$$f(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}, \quad (2.9)$$

dengan $\sigma(x)$ menyatakan fungsi sigmoid. Bentuk perkalian antara x dan $\sigma(x)$ menghasilkan mekanisme *self-gating*, sehingga kontribusi informasi dari suatu neuron dapat diatur secara adaptif (Ramachandran et al., 2017). Grafik fungsi Swish ditunjukkan pada Gambar 2.7.



Gambar 2.7: Grafik fungsi Swish.

2.3.1.4 Teknik Normalisasi

Normalisasi merupakan teknik yang digunakan dalam pelatihan *neural network* untuk menstabilkan distribusi aktivasi antar-*layer* selama proses pelatihan berlangsung (Huang et al., 2020). Cara kerjanya adalah dengan menstandarisasi aktivasi menggunakan rata-rata dan variansi yang dihitung dari kelompok aktivasi tertentu. Hasil normalisasi kemudian disesuaikan kembali menggunakan parameter skala γ dan *shift* β yang dapat dilatih, sehingga model tetap memiliki fleksibilitas dalam mempelajari pola data. Pada penelitian ini, terdapat dua metode normalisasi yang digunakan, yaitu *Layer Normalization* dan *Group Normalization*, yang dijelaskan sebagai berikut.

1. *Layer Normalization*

Layer Normalization (LN) bekerja dengan menghitung rata-rata dan variansi dari seluruh unit dalam satu *layer* untuk setiap sampel secara independen (Ba et al., 2016). Nilai-nilai tersebut kemudian digunakan untuk menstandarisasi aktivasi, sehingga distribusi aktivasi pada setiap *layer* tetap stabil selama pelatihan. Secara matematis, rata-rata μ^{LN} dan standar deviasi σ^{LN} pada LN dihitung sebagai:

$$\mu^{\text{LN}} = \frac{1}{d_h} \sum_{i=1}^{d_h} a_i, \quad \sigma^{\text{LN}} = \sqrt{\frac{1}{d_h} \sum_{i=1}^{d_h} (a_i - \mu^{\text{LN}})^2}, \quad (2.10)$$

dengan d_h adalah jumlah unit dalam *layer* dan a_i adalah aktivasi unit ke- i . Selanjutnya, aktivasi yang telah dinormalisasi ditransformasikan menggunakan parameter yang dapat dipelajari, yaitu:

$$\tilde{a}_i = \frac{\gamma^{\text{LN}}}{\sigma^{\text{LN}}} (a_i - \mu^{\text{LN}}) + \beta^{\text{LN}}, \quad (2.11)$$

dengan γ^{LN} dan β^{LN} masing-masing merupakan parameter skala dan bias yang dipelajari selama pelatihan.

2. *Group Normalization*

Group Normalization (GN) merupakan metode normalisasi yang membagi dimensi fitur dalam suatu *layer* ke dalam G kelompok berukuran sama (Wu & He, 2018). GN menghitung rata-rata dan variansi secara independen untuk setiap kelompok pada setiap sampel. Perhitungan tersebut tidak bergantung pada ukuran *batch*, sehingga GN tetap stabil meskipun *batch size* yang digunakan kecil.

Misalkan C menyatakan jumlah total dimensi fitur dan G adalah jumlah kelompok

yang diinginkan, maka setiap kelompok memuat C/G dimensi. Untuk elemen ke- i dengan indeks *batch* i_N dan indeks dimensi i_C , GN mendefinisikan himpunan elemen yang dinormalisasi bersama sebagai:

$$S_i^{\text{GN}} = \left\{ k \mid k_N = i_N, \left\lfloor \frac{k_C}{C/G} \right\rfloor = \left\lfloor \frac{i_C}{C/G} \right\rfloor \right\}, \quad (2.12)$$

dengan k_N dan k_C masing-masing adalah indeks *batch* dan indeks dimensi dari elemen k . Persamaan (2.12) menyatakan bahwa dua elemen berada dalam kelompok normalisasi yang sama apabila keduanya berasal dari sampel yang sama ($k_N = i_N$) dan dimensinya masuk dalam rentang kelompok yang sama ($\lfloor k_C/(C/G) \rfloor$). Rata-rata dan variansi kemudian dihitung dari himpunan S_i^{GN} tersebut untuk menstandarisasi setiap elemen.

2.3.1.5 Fungsi Loss

Fungsi *loss* merupakan ukuran kuantitatif yang digunakan untuk menilai seberapa jauh hasil prediksi model menyimpang dari nilai target sebenarnya. Dalam konteks *deep learning*, fungsi *loss* digunakan sebagai acuan selama proses pembelajaran, sehingga model dapat memperbarui parameternya untuk menghasilkan prediksi yang lebih baik (Bishop & Nasrabadi, 2006). Pada penelitian ini, terdapat dua fungsi *loss* yang digunakan, yaitu *Binary Cross-Entropy* untuk tugas klasifikasi sentimen dan *Sum of Squared Errors* untuk tugas pendeteksian topik berbasis *clustering*, yang dijelaskan sebagai berikut:

1. Binary Cross-Entropy

Binary Cross-Entropy (BCE) merupakan fungsi *loss* yang umum digunakan pada tugas klasifikasi biner. Dalam penelitian ini, BCE digunakan untuk mengukur kesalahan prediksi pada analisis sentimen dengan dua kelas, yaitu positif dan negatif. BCE sesuai digunakan karena *output* model berupa probabilitas, sehingga selisih antara probabilitas prediksi dan label sebenarnya dapat dihitung secara langsung (Elharrouss et al., 2025). Secara matematis, BCE untuk satu observasi dirumuskan sebagai:

$$\mathcal{L}_{\text{BCE}}(y_i, \hat{y}_i) = -[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)], \quad (2.13)$$

dengan $y_i \in \{0, 1\}$ adalah label asli pada observasi ke- i , dan $\hat{y}_i$ adalah probabilitas prediksi model terhadap kelas positif. Nilai BCE akan semakin kecil apabila probabilitas prediksi model semakin sesuai dengan label sebenarnya. Untuk N

observasi, nilai BCE dapat dituliskan sebagai bentuk rata-rata:

$$\mathcal{L}_{\text{BCE}} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{\text{BCE}}(y_i, \hat{y}_i). \quad (2.14)$$

2. *Sum of Squared Errors*

Sum of Squared Errors (SSE) merupakan fungsi *loss* yang digunakan untuk mengukur jumlah *error* kuadrat antara nilai asli dan nilai hasil prediksi. Secara matematis, SSE dirumuskan sebagai:

$$\mathcal{L}_{\text{SSE}} = \sum_{i=1}^N (y_i - \hat{y}_i)^2, \quad (2.15)$$

dengan y_i adalah nilai asli ke- i , $\hat{y}_i$ adalah nilai hasil prediksi ke- i , dan N adalah jumlah data. Semakin kecil nilai SSE, semakin kecil pula selisih antara nilai asli dan nilai hasil prediksi. Pada metode *clustering*, seperti *K-Means* dan *Deep Embedded K-Means* (DEKM), SSE digunakan untuk mengukur jumlah kuadrat jarak antara setiap data dengan *centroid* dari *cluster* yang sesuai.

3. *Kullback–Leibler Divergence*

Kullback–Leibler (KL) Divergence merupakan fungsi *loss* yang mengukur perbedaan antara dua distribusi probabilitas, yaitu distribusi target P dan distribusi prediksi Q (Goodfellow et al., 2016). Secara matematis, *KL Divergence* didefinisikan sebagai

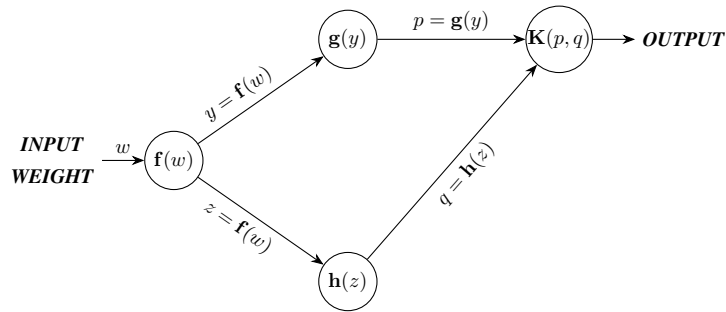
$$\mathcal{L}_{\text{KL}} = \sum_{i=1}^N P(i) \log \frac{P(i)}{Q(i)}, \quad (2.16)$$

dengan $P(i)$ adalah distribusi probabilitas target dan $Q(i)$ adalah distribusi probabilitas hasil prediksi model untuk kelas ke- i . Nilai *KL Divergence* selalu bernilai non-negatif dan bernilai nol apabila kedua distribusi identik. Pada penelitian ini, *KL Divergence* digunakan sebagai fungsi *loss* pada tahap pra-pelatihan xLSTM untuk mengukur kedekatan hasil prediksi model dengan sebaran kata dari dokumen.

2.3.1.6 Backpropagation

Pada *neural network*, proses pelatihan diawali dengan *forward propagation*, di mana *input* x diproses melalui beberapa *hidden layer* hingga menghasilkan *output* o . Berdasarkan *output* tersebut, nilai fungsi *loss* $\mathcal{L}$ dihitung untuk mengevaluasi kesesuaian antara prediksi model dan label sebenarnya. Selanjutnya, dilakukan *backward propagation* atau *backpropagation*, yaitu proses propagasi *error* dari *layer output* ke *layer* sebelumnya untuk menghitung gradien fungsi *loss* terhadap bobot jaringan (Aggarwal et al., 2018). Gradien dihitung karena menunjukkan arah dan besar pengaruh perubahan bobot terhadap nilai *loss*. Informasi ini digunakan dalam proses optimasi untuk memperbarui bobot, sehingga nilai *loss* dapat diminimalkan secara bertahap.

Misalkan terdapat *neural network* dengan beberapa jalur yang menghubungkan *input* ke *output* o sebagaimana ditunjukkan pada Gambar 2.8. *Output* tersebut diperoleh melalui komposisi beberapa fungsi, sehingga fungsi *loss* bergantung pada bobot di setiap *layer*. Oleh karena itu, perhitungan gradien dilakukan menggunakan aturan rantai (*chain rule*) untuk menelusuri kontribusi setiap parameter terhadap *loss*.



Gambar 2.8: Ilustrasi graf komputasi MLP (Aggarwal et al., 2018), telah diolah kembali.

Misalkan w menyatakan suatu bobot pada *layer* awal yang memengaruhi *output* o melalui dua jalur berbeda. Dengan menggunakan aturan rantai, gradien fungsi *loss* terhadap w diperoleh sebagai penjumlahan kontribusi dari setiap jalur, yaitu:

$$\frac{\partial o}{\partial w} = \underbrace{\frac{\partial K(p, q)}{\partial p} g'(y) f'(w)}_{\text{jalur pertama}} + \underbrace{\frac{\partial K(p, q)}{\partial q} h'(z) f'(w)}_{\text{jalur kedua}}, \quad (2.17)$$

dengan $p = g(y)$ dan $q = h(z)$ merupakan hasil komposisi fungsi pada masing-masing jalur, sedangkan y dan z adalah *output* dari neuron pada *layer* sebelumnya.

Secara umum, misalkan suatu *neural network* memiliki *hidden unit* $h_1, h_2, \dots, h_k$ yang terhubung ke *output* o , dengan bobot w_r yang menghubungkan unit h_{r-1} ke h_r .

Misalkan pula $\mathcal{P}$ adalah himpunan semua jalur dari h_r menuju o , maka gradien fungsi *loss* $\mathcal{L}$ terhadap bobot tersebut dirumuskan sebagai:

$$\frac{\partial \mathcal{L}}{\partial w_{(h_{r-1}, h_r)}} = \frac{\partial \mathcal{L}}{\partial o} \left[\sum_{p \in \mathcal{P}} \frac{\partial o}{\partial h_k} \prod_{i=r}^{k-1} \frac{\partial h_{i+1}}{\partial h_i} \right] \frac{\partial \mathcal{L}}{\partial w_{(h_{r-1}, h_r)}}. \quad (2.18)$$

Untuk menyederhanakan notasi, ekspresi dalam tanda kurung pada Persamaan (2.18) didefinisikan sebagai $\Delta(h_r, o) = \frac{\partial \mathcal{L}}{\partial h_r}$, sehingga gradien lokal terhadap bobot dapat dituliskan sebagai:

$$\frac{\partial h_r}{\partial w_{(h_{r-1}, h_r)}} = f'_r(z_r) h_{r-1}. \quad (2.19)$$

Karena perhitungan $\Delta(h_r, o)$ melibatkan penjumlahan atas seluruh jalur pada jaringan, pendekatan rekursif digunakan agar gradien dapat dihitung secara efisien dari *layer output* menuju *layer-layer* sebelumnya. Proses ini diinisialisasi dengan $\Delta(o, o) = \frac{\partial \mathcal{L}}{\partial o}$, kemudian disebarkan menggunakan hubungan rekursif berikut:

$$\Delta(h_r, o) = \sum_{h_{r+1} \in \mathbf{h}_{r+1}} f'_{r+1}(z_{r+1}) w_{r+1} \Delta(h_{r+1}, o). \quad (2.20)$$

Gradien fungsi *loss* terhadap setiap bobot kemudian diperoleh dengan mengombinasikan $\Delta(h_r, o)$ dari Persamaan (2.20) dan gradien lokal dari Persamaan (2.19), yang selanjutnya digunakan untuk memperbarui parameter pada setiap iterasi pelatihan.

2.3.1.7 Optimasi Adam with Weight Decay

Proses optimasi merupakan tahapan dalam model *deep learning* yang dapat dipandang sebagai pencarian parameter untuk menghasilkan nilai fungsi *loss* sekecil mungkin selama pelatihan (Zhang et al., 2023). Dalam pelatihan *neural network*, terdapat berbagai metode optimasi yang umum digunakan, seperti *Stochastic Gradient Descent* (SGD), RMSProp, Adam, dan AdamW. Pada penelitian ini, metode optimasi yang digunakan adalah AdamW.

Adaptive Moment Estimation (Adam) merupakan algoritma optimasi yang mengombinasikan konsep *momentum* dengan penyesuaian *learning rate* secara adaptif (Kingma & Ba, 2017). Pada setiap iterasi ke- t , Adam mendeteksi pergerakan rata-rata (*moving average*) dari gradien orde pertama dan orde kedua sebagai dasar pembaruan parameter. Misalkan $\mathbf{g}^{(t)} = \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}^{(t-1)})$ menyatakan gradien fungsi *loss* pada iterasi ke- t , maka estimasi momen pertama dan momen kedua dihitung sebagai berikut:

$$\mathbf{m}^{(t)} = \beta_1 \mathbf{m}^{(t-1)} + (1 - \beta_1) \mathbf{g}^{(t)}, \quad (2.21)$$

$$\mathbf{v}^{(t)} = \beta_2 \mathbf{v}^{(t-1)} + (1 - \beta_2) (\mathbf{g}^{(t)})^2, \quad (2.22)$$

dengan β_1 dan β_2 merupakan koefisien peluruhan untuk masing-masing momen. Karena kedua estimasi diinisialisasi dengan nol, diperlukan koreksi bias agar tidak cenderung menuju nol pada iterasi awal:

$$\hat{\mathbf{m}}^{(t)} = \frac{\mathbf{m}^{(t)}}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}^{(t)} = \frac{\mathbf{v}^{(t)}}{1 - \beta_2^t}. \quad (2.23)$$

Parameter kemudian diperbarui menggunakan:

$$\mathbf{w}^{(t)} = \mathbf{w}^{(t-1)} - \eta \frac{\hat{\mathbf{m}}^{(t)}}{\sqrt{\hat{\mathbf{v}}^{(t)} + \varepsilon}}, \quad (2.24)$$

dengan η menyatakan *learning rate* dan ε merupakan konstanta kecil untuk mencegah pembagian dengan nol. Meskipun efektif, Adam memiliki kelemahan dalam penanganan regularisasi.

Pada Adam standar, regularisasi ditambahkan langsung ke dalam gradien saat pembaruan parameter, sehingga efek penalti pada bobot bercampur dengan mekanisme adaptif gradien dan menjadi kurang konsisten antar parameter. Untuk mengatasi hal ini, *Adaptive Moment Estimation with Weight Decay* (AdamW) memisahkan penerapan *weight decay* dari pembaruan parameter, sehingga penalti terhadap bobot diterapkan secara langsung (Loshchilov & Hutter, 2017). Pembaruan parameter pada AdamW dirumuskan sebagai:

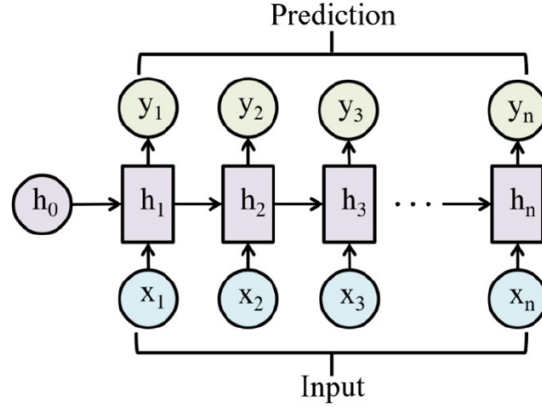
$$\mathbf{w}^{(t)} = \mathbf{w}^{(t-1)} - \eta \left(\frac{\alpha \hat{\mathbf{m}}^{(t)}}{\sqrt{\hat{\mathbf{v}}^{(t)} + \varepsilon}} + \lambda \mathbf{w}^{(t-1)} \right), \quad (2.25)$$

dengan α menyatakan *learning rate* dan λ merupakan koefisien *weight decay*. Pemisahan ini membuat proses regularisasi bekerja terpisah dari adaptasi gradien, sehingga AdamW mampu memberikan kemampuan generalisasi yang lebih baik dibandingkan Adam.

2.4 Recurrent Neural Network

Recurrent Neural Network (RNN) merupakan arsitektur *neural network* untuk memodelkan data berurutan, seperti teks dan deret waktu (Mienye et al., 2024). RNN memiliki koneksi berulang yang memungkinkan informasi dari langkah sebelumnya dipertahankan, sehingga mampu menangkap ketergantungan waktu antarelelemen

(Schmidt, 2019). Pada setiap waktu t , *input* x_t diproses untuk menghasilkan *hidden state* h_t sebagai representasi informasi hingga waktu tersebut, yang kemudian digunakan untuk menghasilkan *output* $\hat{y}_t$.



Gambar 2.9: Arsitektur RNN (Mienye et al., 2024).

Arsitektur dasar RNN ditunjukkan pada Gambar 2.9. Koneksi dari h_{t-1} ke h_t menunjukkan bahwa pemrosesan pada setiap langkah waktu dipengaruhi oleh *input* saat ini sekaligus konteks dari langkah sebelumnya. Misalkan urutan *input* $\{x_t\}_{t=1}^T$, *hidden state* $\{h_t\}_{t=1}^T$, dan *output* $\{\hat{y}_t\}_{t=1}^T$, maka pembaruan *hidden state* dirumuskan sebagai

$$h_t = f(w_x x_t + w_h h_{t-1} + b_h), \quad (2.26)$$

dengan w_x sebagai bobot dari *input* ke *hidden layer*, w_h sebagai bobot *recurrent*, b_h sebagai bias, dan $f(\cdot)$ sebagai fungsi aktivasi nonlinier. Nilai awal h_0 umumnya diinisialisasi sebagai vektor nol. Selanjutnya, *output* dihitung dari *hidden state* sebagai

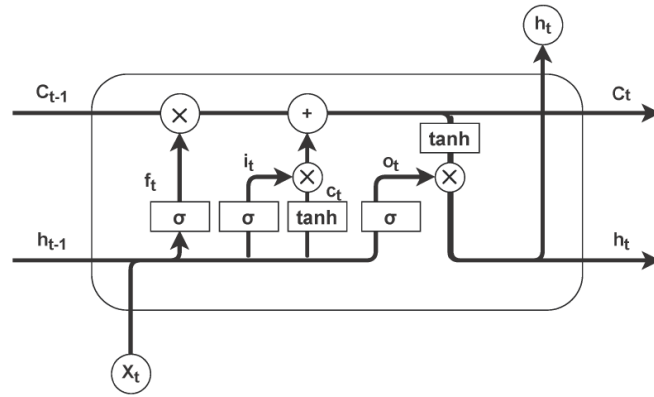
$$\hat{y}_t = f(w_o h_t + b_o), \quad (2.27)$$

dengan w_o sebagai bobot dari *hidden layer* ke *output* dan b_o sebagai bias. Persamaan (2.26) dan (2.27) bersama-sama menggabungkan informasi saat ini dan konteks masa lalu untuk menghasilkan prediksi.

Meskipun efektif dalam memodelkan data berurutan, RNN menghadapi kendala *vanishing* dan *exploding gradients*, terutama pada ketergantungan jangka panjang. Hal ini disebabkan oleh propagasi gradien yang melibatkan perkalian berulang sepanjang urutan, sehingga nilai gradien dapat mengecil atau membesar secara drastis. Akibatnya, model kesulitan mempelajari hubungan jangka panjang, sehingga dikembangkan varian seperti LSTM (Schmidt, 2019).

2.5 Long Short-Term Memory

Long Short-Term Memory (LSTM) merupakan pengembangan dari RNN yang dirancang untuk mengatasi keterbatasan RNN dalam mempelajari dependensi jangka panjang. LSTM memperkenalkan *cell state* sebagai jalur utama penyimpanan informasi jangka panjang, serta *gating mechanism* untuk mengendalikan informasi yang perlu disimpan, diperbarui, atau dihapus pada setiap langkah waktu (Mienye et al., 2024). Ketiga *gate* ini berkontribusi untuk mengatur informasi yang boleh masuk, dihapus, dan yang diteruskan sebagai *output*, sehingga LSTM dapat mempertahankan informasi yang relevan meskipun urutan teksnya panjang. Ilustrasi arsitektur LSTM pada satu langkah waktu ditunjukkan pada Gambar 2.10. Pada ilustrasi tersebut *input* x_t , *hidden state* sebelumnya h_{t-1} , dan *cell state* sebelumnya c_{t-1} digunakan bersama untuk menghasilkan *cell state* c_t dan *hidden state* h_t pada waktu t (Mienye et al., 2024).



Gambar 2.10: Arsitektur dasar LSTM (Mienye et al., 2024).

LSTM memiliki tiga *gate* utama yang bekerja pada setiap langkah waktu t , yaitu *forget gate* f_t yang memutuskan informasi mana dari *cell state* sebelumnya yang perlu dibuang, *input gate* i_t yang mengatur seberapa besar informasi baru z_t yang dimasukkan ke memori, dan *output gate* o_t yang menentukan bagian mana dari *cell state* yang diteruskan sebagai *hidden state*. Operasi keseluruhan LSTM pada waktu ke- t dirumuskan sebagai berikut:

$$f_t = \sigma(\tilde{f}_t), \quad \tilde{f}_t = \mathbf{w}_f^\top \mathbf{x}_t + r_f h_{t-1} + b_f, \quad (2.28)$$

$$i_t = \sigma(\tilde{i}_t), \quad \tilde{i}_t = \mathbf{w}_i^\top \mathbf{x}_t + r_i h_{t-1} + b_i, \quad (2.29)$$

$$z_t = \tanh(\tilde{z}_t), \quad \tilde{z}_t = \mathbf{w}_z^\top \mathbf{x}_t + r_z h_{t-1} + b_z, \quad (2.30)$$

$$c_t = f_t c_{t-1} + i_t z_t, \quad (2.31)$$

$$o_t = \sigma(\tilde{o}_t), \quad \tilde{o}_t = \mathbf{w}_o^\top \mathbf{x}_t + r_o h_{t-1} + b_o, \quad (2.32)$$

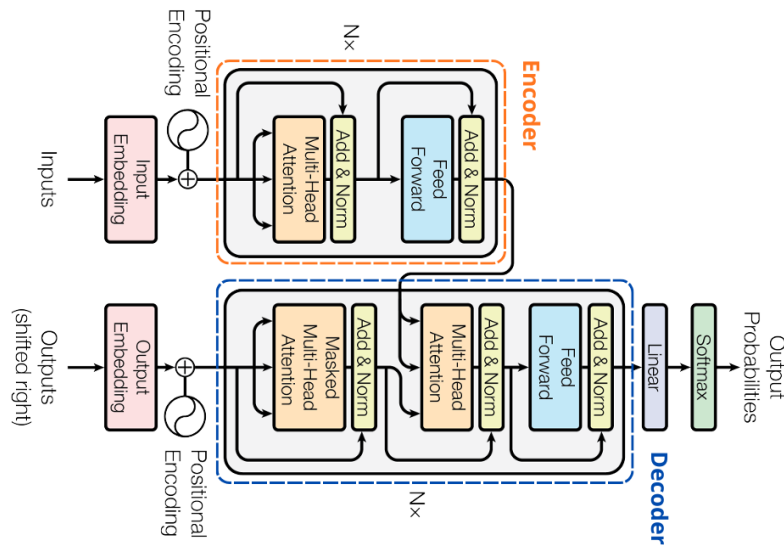
$$h_t = o_t \tilde{h}_t, \quad \tilde{h}_t = \tanh(c_t). \quad (2.33)$$

Pada setiap persamaan *gate*, pra-aktivasi $(\tilde{\cdot})$ dihitung dari kombinasi linear antara *input* saat ini $\mathbf{x}_t$, *hidden state* sebelumnya h_{t-1} , dan bias b , yang kemudian diproses dengan fungsi aktivasi sigmoid $\sigma(\cdot)$. Persamaan (2.31) menjadi bagian utama LSTM dalam memperbarui memori. *Cell state* diperbarui dengan tetap mempertahankan sebagian informasi lama yang diatur oleh *forget gate* f_t , sekaligus menambahkan informasi baru yang dikendalikan oleh *input gate* i_t . Setelah itu, *hidden state* h_t dihasilkan dengan membatasi *cell state* melalui *output gate* o_t .

2.6 Transformer

Penjelasan pada subbab ini mengacu pada penelitian (Vaswani et al., 2023), kecuali jika bersumber dari referensi lain. Transformer merupakan arsitektur *neural network* untuk memodelkan data berurutan yang memanfaatkan mekanisme *attention* tanpa menggunakan koneksi rekuren ataupun konvolusional. Mekanisme ini memungkinkan model menilai hubungan antarelemen dalam suatu urutan melalui pemberian bobot yang berbeda. Informasi yang relevan kemudian dapat diproses secara selektif, sementara setiap elemen dapat berinteraksi secara langsung tanpa bergantung pada pemrosesan berurutan. Transformer menunjukkan keunggulan pada berbagai bidang seperti *language modeling* (Radford & Narasimhan, 2018), peramalan deret waktu (Nie et al., 2023), dan *computer vision* (Dosovitskiy et al., 2021).

2.6.1 Arsitektur Transformer



Gambar 2.11: Arsitektur dasar Transformer (Vaswani et al., 2023), telah diolah kembali.

Arsitektur Transformer tersusun atas *encoder* dan *decoder*, yang masing-masing terdiri dari tumpukan beberapa *layer* identik, sebagaimana ditunjukkan pada Gambar 2.11. *Encoder* berfungsi memetakan urutan *input* ke dalam representasi laten yang bersifat kontekstual, sedangkan *decoder* menghasilkan urutan *output* secara autoregresif berdasarkan representasi tersebut. Setiap *layer* pada *encoder* dan *decoder* dilengkapi dengan mekanisme *attention*, transformasi nonlinier, *skip connection*, dan *layer normalization* untuk menjaga kestabilan aliran gradien selama pelatihan.

2.6.2 Word Embedding

Dalam pemrosesan teks, token tidak dapat digunakan secara langsung dalam bentuk simbol diskrit, sehingga diperlukan representasi numerik agar dapat digunakan sebagai *input* model *deep learning*. Pendekatan yang umum digunakan adalah *word embedding*, yaitu pemetaan token ke dalam vektor kontinu berdimensi tetap yang mampu menangkap informasi semantik berdasarkan kemunculan kata dalam konteks tertentu (Almeida & Xexéo, 2023). Representasi vektor ini memungkinkan kata-kata dengan makna atau penggunaan yang serupa memiliki representasi yang saling berdekatan dalam ruang vektor.

Pada arsitektur Transformer, setiap token pada posisi ke- t direpresentasikan sebagai vektor *embedding* $e_t \in \mathbb{R}^{d_{\text{model}}}$. Urutan *input* $(x_1, x_2, \dots, x_n)$ kemudian diubah menjadi

urutan vektor kontinu $(e_1, e_2, \dots, e_n)$. Representasi ini selanjutnya digunakan sebagai *input* ke bagian *encoder*.

2.6.3 Encoder

Encoder merupakan komponen dalam arsitektur Transformer yang bertujuan memetakan urutan token *input* ke dalam representasi vektor yang bersifat kontekstual. Secara struktural, *encoder* tersusun atas tumpukan L layer identik, di mana setiap layer terdiri dari dua sub-layer utama, yaitu *multi-head self-attention* dan *position-wise feed-forward network*. Setiap sub-layer dilengkapi dengan *skip connection* dan *layer normalization*, sehingga *output layer* dinyatakan sebagai $\text{LayerNorm}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$. Seluruh sub-layer menghasilkan representasi dengan dimensi yang sama, yaitu d_{model} , agar *skip connection* dapat diterapkan secara konsisten. *Output* dari layer *encoder* terakhir berupa sekumpulan vektor representasi kontekstual yang selanjutnya digunakan sebagai *input* bagi *decoder*.

2.6.4 Mekanisme Attention

Mekanisme *attention* memungkinkan model memusatkan pemrosesan pada bagian-bagian *input* yang paling relevan terhadap suatu tugas tertentu. Model sekuensial umumnya memadatkan informasi dari suatu urutan ke dalam satu vektor tetap. Sementara itu, *attention* memberikan bobot adaptif pada setiap elemen *input*, sehingga setiap token dapat berinteraksi langsung dengan token lain dalam urutan (Luong et al., 2015).

2.6.4.1 Scaled Dot-Product Attention

Mekanisme *scaled dot-product attention* menentukan relevansi antar token melalui tiga komponen utama, yaitu *query* (Q), *key* (K), dan *value* (V). *Query* merepresentasikan informasi yang dicari, *key* merepresentasikan informasi pembanding, sedangkan *value* merepresentasikan informasi yang digabungkan berdasarkan relevansi *query* dan *key*. Ketiga matriks ini diperoleh melalui proyeksi linier dari matriks *input* $X \in \mathbb{R}^{n \times d_{\text{model}}}$:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V, \quad (2.34)$$

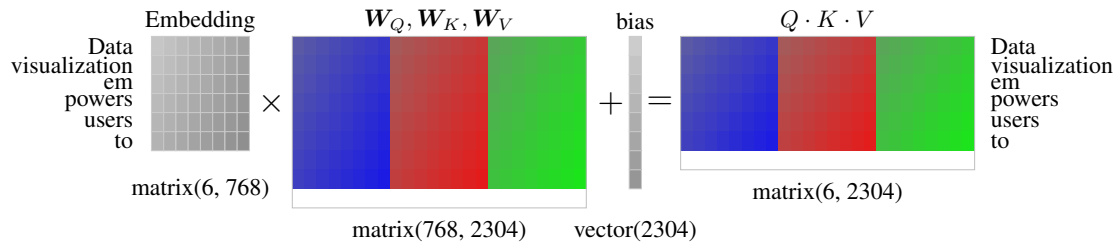
dengan $W^Q, W^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ dan $W^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ merupakan matriks bobot yang dioptimalkan selama pelatihan. Skor relevansi dihitung melalui perkalian titik antara *query* dan *key*, kemudian diskalakan dengan $\sqrt{d_k}$ untuk menjaga kestabilan gradien. Skor

tersebut selanjutnya dinormalisasi menggunakan fungsi *softmax* dan digunakan untuk membobotkan matriks *value*:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (2.35)$$

2.6.4.2 Multi-Head Attention

Multi-head attention merupakan pengembangan dari mekanisme *scaled dot-product attention* yang memungkinkan model memproses informasi secara paralel melalui beberapa *head*. Setiap *head* menggunakan parameter proyeksi yang berbeda sehingga dapat menangkap pola hubungan yang berbeda antar token secara simultan. Ilustrasi proses pembentukan $\mathbf{Q}$, $\mathbf{K}$, dan $\mathbf{V}$ dari *input embedding* ditunjukkan pada Gambar 2.12.



Gambar 2.12: Ilustrasi proses pembentukan *query*, *key*, dan *value* dari *input embedding* (Pradipta Hardinatha et al., 2025), telah diolah kembali.

Secara matematis, untuk setiap *head* ke- h :

$$\mathbf{Q}^{(h)} = \mathbf{Q}\mathbf{W}_h^Q, \quad \mathbf{K}^{(h)} = \mathbf{K}\mathbf{W}_h^K, \quad \mathbf{V}^{(h)} = \mathbf{V}\mathbf{W}_h^V, \quad (2.36)$$

dengan *output* setiap *head* diperoleh melalui $\text{head}_h = \text{Attention}(\mathbf{Q}^{(h)}, \mathbf{K}^{(h)}, \mathbf{V}^{(h)})$. Seluruh *head* kemudian digabungkan dan diproyeksikan kembali:

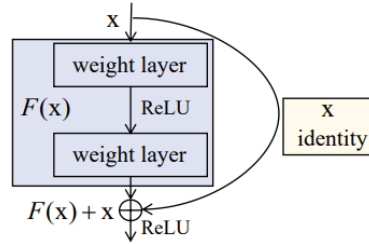
$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \mathbf{W}^O. \quad (2.37)$$

2.6.5 Skip Connection

Skip connection atau *residual connection* merupakan mekanisme yang memungkinkan *input* suatu *layer* diteruskan secara langsung ke *layer* berikutnya tanpa harus sepenuhnya melalui transformasi nonlinier. Secara umum, jika suatu sub-layer direpresentasikan sebagai fungsi $\mathcal{F}(\cdot)$ dengan *input* $\mathbf{x}$, maka *output* residual dapat dirumuskan sebagai

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}. \quad (2.38)$$

Pada *skip connection*, model mempelajari representasi baru $\mathcal{F}(x)$ dan mempertahankan informasi awal dari x . Jaringan cukup mempelajari perubahan atau koreksi terhadap *input* awal, sehingga proses pelatihan menjadi lebih stabil dan membantu mengurangi permasalahan hilangnya gradien pada jaringan yang dalam (Xu et al., 2025). Ilustrasi mekanisme *skip connection* ditunjukkan pada Gambar 2.13.



Gambar 2.13: Ilustrasi *skip connection* (Xu et al., 2025).

2.6.6 Decoder

Decoder merupakan komponen dalam arsitektur Transformer yang bertugas menghasilkan urutan *output* berdasarkan representasi kontekstual dari *encoder*. *Decoder* bekerja secara autoregresif, yaitu setiap prediksi token bergantung pada token-token *output* sebelumnya serta informasi dari *encoder*. Secara struktural, *decoder* terdiri atas tumpukan L layer identik. Setiap layer memiliki tiga sub-layer utama, yaitu *masked multi-head self-attention*, *encoder-decoder attention*, dan *position-wise feed-forward network*. Masing-masing sub-layer tersebut dilengkapi dengan *skip connection* dan *layer normalization*.

Pada sub-layer *masked self-attention*, pembatasan akses terhadap token masa depan diterapkan menggunakan matriks *look-ahead mask* M , sehingga prediksi token ke- t hanya bergantung pada token-token sebelumnya. Secara matematis, mekanisme ini dirumuskan sebagai:

$$\text{MaskedAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right) \mathbf{V}, \quad (2.39)$$

dengan M berisi nilai 0 pada posisi yang diperbolehkan dan $-\infty$ pada posisi yang dibatasi, sehingga token masa depan tidak berkontribusi dalam perhitungan *attention* setelah normalisasi *softmax*.

2.7 Clustering

Clustering merupakan metode dalam *unsupervised learning* yang bertujuan mengelompokkan data ke dalam beberapa *cluster* berdasarkan tingkat kemiripan tanpa menggunakan label kelas (Jain et al., 1999). Setiap *cluster* diharapkan memiliki kemiripan yang tinggi antar anggotanya. Selain itu, setiap *cluster* juga perlu memiliki perbedaan yang jelas dengan *cluster* lain, sehingga metode ini berguna untuk menemukan struktur atau pola tersembunyi pada data, terutama data berdimensi tinggi (Bishop & Nasrabadi, 2006).

Berdasarkan keanggotaan data, *clustering* dapat dibedakan menjadi *hard clustering* dan *soft clustering* (Petrus et al., 2019). Pada *hard clustering*, setiap data hanya menjadi anggota satu *cluster*, sedangkan pada *soft clustering*, data dapat memiliki derajat keanggotaan pada lebih dari satu *cluster*. Salah satu metode *hard clustering* yang paling umum digunakan adalah *K-Means clustering*. Dalam penerapannya, data dibagi ke dalam K *cluster* menggunakan *centroid* sebagai pusat kelompok. Pembagian tersebut dilakukan dengan meminimalkan jarak antara setiap titik data dan pusat *cluster* yang merepresentasikannya (Bishop & Nasrabadi, 2006).

Kedekatan data terhadap *centroid* umumnya diukur menggunakan jarak Euclidean (Bishop & Nasrabadi, 2006). Jarak Euclidean antara dua vektor $\mathbf{p}$ dan $\mathbf{q}$ pada ruang berdimensi D didefinisikan sebagai

$$\|\mathbf{p} - \mathbf{q}\| = \sqrt{\sum_{d=1}^D (p_d - q_d)^2}, \quad (2.40)$$

dengan $\mathbf{p} = (p_1, p_2, \dots, p_D)$ dan $\mathbf{q} = (q_1, q_2, \dots, q_D)$. Ukuran ini menjadi dasar *assignment* data ke dalam *cluster* yang *centroid*-nya paling dekat.

Misalkan diberikan himpunan data $\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$ dengan $\mathbf{x}_n \in \mathbb{R}^D$, dan pusat *cluster* $\boldsymbol{\mu}_k \in \mathbb{R}^D$ untuk $k = 1, 2, \dots, K$ yang diinisialisasi secara acak. Tujuan utama *K-Means* adalah meminimalkan fungsi objektif

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2, \quad (2.41)$$

dengan $r_{nk} \in \{0, 1\}$ sebagai indikator keanggotaan *cluster*, di mana $r_{nk} = 1$ apabila $\mathbf{x}_n$ termasuk ke dalam *cluster* ke- k , dan berlaku kendala $\sum_{k=1}^K r_{nk} = 1$ untuk setiap n .

Optimisasi dilakukan secara bergantian antara dua tahap. Pertama, dengan $\boldsymbol{\mu}_k$ tetap, setiap data dimasukkan ke *cluster* dengan *centroid* terdekat:

$$r_{nk} = \begin{cases} 1, & \text{jika } k = \arg \min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2, \\ 0, & \text{lainnya.} \end{cases} \quad (2.42)$$

Kedua, dengan r_{nk} tetap, *centroid* diperbarui dengan menetapkan $\frac{\partial J}{\partial \boldsymbol{\mu}_k} = \mathbf{0}$, sehingga diperoleh:

$$\boldsymbol{\mu}_k = \frac{\sum_{n=1}^N r_{nk} \mathbf{x}_n}{\sum_{n=1}^N r_{nk}}. \quad (2.43)$$

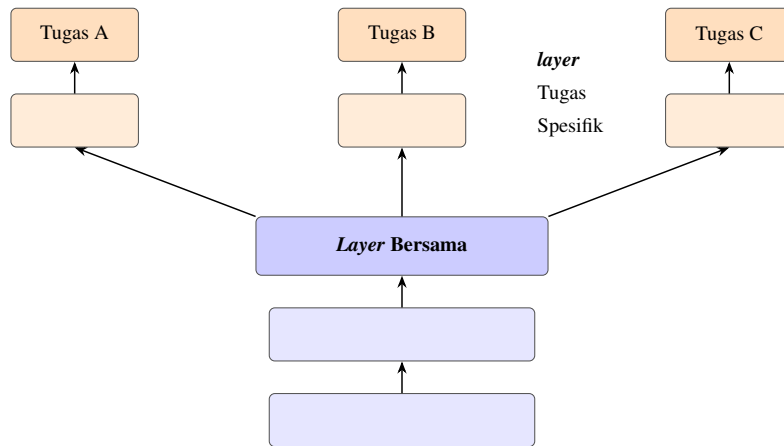
Persamaan (2.43) menunjukkan bahwa *centroid* setiap *cluster* merupakan rata-rata dari seluruh anggotanya. Kedua tahap ini dilakukan berulang sampai perubahan nilai fungsi objektif lebih kecil dari batas toleransi ε atau jumlah iterasi mencapai *maxiter*. Dalam penelitian ini, metode *clustering* yang digunakan adalah *Deep Embedded K-Means* (DEKM) (Guo et al., 2021), yang memperluas cara kerja *K-Means* dengan terlebih dahulu mempelajari representasi laten yang lebih informatif melalui *autoencoder* sebelum melakukan pengelompokan.

2.8 Multi-Task Learning

Penjelasan pada subbab ini mengacu pada penelitian (Ruder, 2017), kecuali jika bersumber dari referensi lain. *Multi-Task Learning* (MTL) merupakan pendekatan pembelajaran yang memungkinkan model mempelajari beberapa tugas secara bersamaan dengan memanfaatkan representasi bersama dari data. Tugas-tugas yang saling berkaitan dapat saling berkontribusi selama proses pembelajaran, sehingga informasi dari satu tugas membantu pembentukan representasi yang lebih baik untuk tugas lainnya.

MTL dalam *deep learning* umumnya diterapkan melalui *hard parameter sharing* dan *soft parameter sharing*. Pendekatan *hard parameter sharing* merupakan salah satu metode yang paling umum digunakan, dengan beberapa *hidden layer* yang dibagikan secara langsung antar tugas dan *output layer* yang spesifik untuk setiap tugas.

Ilustrasi pendekatan ini ditunjukkan pada Gambar 2.14, dengan *layer* awal sebagai *shared layers* dan *layer* atas sebagai *task-specific layer*. Pemisahan antara *layer* bersama dan *layer* spesifik memungkinkan model mempelajari informasi umum sekaligus menyesuaikan pembelajaran untuk masing-masing tugas.



Gambar 2.14: Ilustrasi pendekatan *hard parameter sharing* (Ruder, 2017), telah diolah kembali.

Berbeda dengan *hard parameter sharing*, *soft parameter sharing* menggunakan model terpisah untuk setiap tugas, tetapi tetap menjaga keterkaitan antar model melalui regularisasi parameter. Parameter antar model tidak dibagikan secara langsung, namun dibatasi agar memiliki kemiripan tertentu sehingga informasi dari setiap tugas tetap dapat dimanfaatkan.

Berdasarkan prinsip pembelajaran bersama dalam MTL, penelitian ini menerapkan *hard parameter sharing* untuk mengintegrasikan analisis sentimen dan pendeteksian topik dalam satu model terpadu. Representasi bersama dibentuk menggunakan xLSTM sebagai *encoder* atau *shared layer* untuk memproses teks. Representasi laten yang dihasilkan kemudian digunakan untuk dua tugas, yaitu klasifikasi sentimen dengan *fully connected layer* dan pendeteksian topik dengan DEKM.

2.9 Metrik Evaluasi

Metrik evaluasi merupakan ukuran kuantitatif yang digunakan untuk menilai kinerja suatu model dalam menyelesaikan tugas tertentu berdasarkan kriteria yang telah ditetapkan. Jenis metrik yang digunakan bergantung pada karakteristik tugas dan model yang diterapkan. Pada model klasifikasi, seperti analisis sentimen, evaluasi umumnya dilakukan menggunakan ukuran berbasis *confusion matrix* untuk menilai ketepatan prediksi terhadap kelas positif maupun negatif (Sokolova & Lapalme, 2009). Sementara itu, evaluasi pendeteksian topik yang bersifat *unsupervised* dilakukan menggunakan *topic coherence* dan *topic diversity*. *Topic coherence* menilai keterkaitan semantik antar kata dalam satu topik (O'Callaghan et al., 2015), sedangkan *topic diversity* menilai keberagaman kata antar topik yang dihasilkan (Dieng et al., 2019).

2.9.1 Metrik Evaluasi Analisis Sentimen

Pada tugas analisis sentimen, evaluasi kinerja model dilakukan untuk menilai kemampuan model dalam membedakan teks bersentimen positif dan negatif. *Confusion matrix* digunakan untuk menunjukkan hubungan antara hasil prediksi model dan label aktual. Tabel ini menjadi dasar perhitungan berbagai metrik evaluasi klasifikasi, seperti *accuracy*, *precision*, *recall*, dan *F1-score* (Sokolova & Lapalme, 2009). Ilustrasi *confusion matrix* untuk kasus klasifikasi biner ditunjukkan pada Tabel 2.1.

Tabel 2.1: *Ilustrasi Confusion Matrix*

	Prediksi Positif	Prediksi Negatif
Aktual Positif	<i>True Positive (TP)</i>	<i>False Negative (FN)</i>
Aktual Negatif	<i>False Positive (FP)</i>	<i>True Negative (TN)</i>

Setiap elemen pada *confusion matrix* memiliki makna yang penting dalam menentukan performa model klasifikasi. Penjelasan mengenai masing-masing komponen pada *confusion matrix* disajikan sebagai berikut:

- *True Positive (TP)* : jumlah data positif yang berhasil diprediksi benar sebagai positif oleh model.
- *True Negative (TN)* : jumlah data negatif yang berhasil diprediksi benar sebagai negatif oleh model.
- *False Positive (FP)* : jumlah data negatif yang salah diprediksi sebagai positif (dikenal sebagai *Type I error*).
- *False Negative (FN)* : jumlah data positif yang salah diprediksi sebagai negatif (dikenal sebagai *Type II error*).

Berdasarkan keempat komponen tersebut, sejumlah metrik evaluasi dapat digunakan untuk menilai performa model dalam melakukan klasifikasi sentimen secara kuantitatif. Metrik yang paling umum digunakan meliputi *accuracy*, *precision*, *recall*, dan *F1-score*, yang masing-masing dijelaskan sebagai berikut:

1. *Accuracy*

Accuracy mengukur proporsi total prediksi yang benar terhadap seluruh data uji. Metrik ini memberikan gambaran umum mengenai kinerja model klasifikasi, namun kurang representatif pada kondisi distribusi kelas yang tidak seimbang

karena prediksi terhadap kelas mayoritas dapat mendominasi nilai akurasi (Sokolova & Lapalme, 2009). Secara matematis, *accuracy* dirumuskan sebagai

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}. \quad (2.44)$$

2. *Precision*

Precision menunjukkan proporsi prediksi positif yang benar dibandingkan dengan seluruh prediksi positif yang dihasilkan oleh model. Metrik ini penting untuk menilai kemampuan model dalam meminimalkan kesalahan *false positive*, khususnya pada kasus di mana kesalahan klasifikasi positif memiliki konsekuensi yang signifikan (Sokolova & Lapalme, 2009). *Precision* didefinisikan sebagai

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}. \quad (2.45)$$

3. *Recall*

Recall mengukur kemampuan model dalam mendeteksi seluruh data yang benar-benar termasuk dalam kelas positif. Nilai *recall* yang tinggi menunjukkan sensitivitas model terhadap kelas positif, namun dalam praktiknya sering kali dicapai dengan peningkatan jumlah *false positive* (Sokolova & Lapalme, 2009). *Recall* dirumuskan sebagai

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \quad (2.46)$$

4. *F1-score*

F1-score merupakan rata-rata harmonik antara *precision* dan *recall*, yang dirancang untuk memberikan penilaian seimbang terhadap performa model klasifikasi. Metrik ini umum digunakan pada kondisi data tidak seimbang karena mampu merepresentasikan kompromi antara ketepatan prediksi positif dan kemampuan model dalam mendeteksi kelas positif (Sokolova & Lapalme, 2009). *F1-score* didefinisikan sebagai

$$F_1\text{-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (2.47)$$

2.9.2 Metrik Evaluasi pendeteksian topik

Penelitian ini menggunakan metrik evaluasi berupa *topic coherence* dan *topic diversity* untuk menilai kualitas topik yang dihasilkan oleh model. *Topic coherence* digunakan untuk mengevaluasi keterkaitan semantik antar kata dalam satu topik, sedangkan *topic diversity* digunakan untuk mengevaluasi keberagaman kata antar topik. Penjelasan mengenai kedua metrik tersebut disajikan sebagai berikut:

1. Topic Coherence

Topic coherence merupakan ukuran yang digunakan untuk menilai sejauh mana kata-kata dalam satu topik saling berkaitan secara semantik dan membentuk makna yang mudah diinterpretasikan (Röder et al., 2015).

Dalam penelitian ini, digunakan *topic coherence* berbasis *word embedding*, yaitu TC-W2V, yang mengukur kesamaan semantik antar kata menggunakan representasi vektor kata (O’Callaghan et al., 2015). Perhitungan TC-W2V dilakukan dengan merepresentasikan setiap kata dalam topik ke dalam bentuk vektor menggunakan model *word embedding* seperti *word2vec*.

Selanjutnya, untuk setiap pasangan kata dalam himpunan topik $W = \{\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_N\}$, dihitung kesamaan semantik menggunakan *cosine similarity*, sebagaimana dirumuskan pada Persamaan (2.48).

$$\text{similarity}(\mathbf{w}_i, \mathbf{w}_j) = \frac{\mathbf{w}_i^\top \mathbf{w}_j}{\|\mathbf{w}_i\| \|\mathbf{w}_j\|}, \quad (2.48)$$

Nilai *topic coherence* kemudian diperoleh dengan menghitung rata-rata kesamaan semantik dari seluruh pasangan kata dalam topik, sebagaimana dirumuskan pada Persamaan (2.49).

$$TC-W2V = \frac{2}{N(N-1)} \sum_{j=2}^N \sum_{i=1}^{j-1} \text{similarity}(\mathbf{w}_i, \mathbf{w}_j), \quad (2.49)$$

dengan $\text{similarity}(\mathbf{w}_i, \mathbf{w}_j)$ menyatakan similaritas antara vektor kata $\mathbf{w}_i$ dan $\mathbf{w}_j$. Nilai yang lebih tinggi menunjukkan bahwa kata-kata dalam topik memiliki keterkaitan semantik yang lebih kuat, sehingga topik yang dihasilkan lebih koheren dan mudah diinterpretasikan.

2. Topic Diversity

Topic diversity merupakan ukuran yang digunakan untuk menilai keberagaman kata antar topik. Metrik ini menghitung proporsi kata unik dari daftar kata teratas pada seluruh topik (Dieng et al., 2019). Secara matematis, *topic diversity* dirumuskan pada Persamaan (2.50).

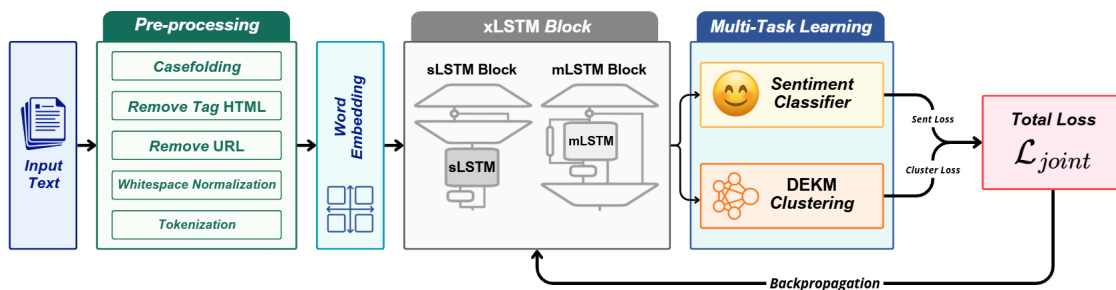
$$Topic\ Diversity = \frac{\text{Jumlah kata unik pada seluruh topik}}{K \times N}, \quad (2.50)$$

dengan K menyatakan jumlah topik atau *cluster* dan N menyatakan jumlah kata teratas yang digunakan pada setiap topik. Nilai *topic diversity* yang semakin tinggi menunjukkan bahwa kata-kata yang muncul pada setiap topik semakin beragam, sehingga topik yang dihasilkan cenderung tidak redundan.

BAB 3

METODE *JOINT DOCUMENT LEVEL SENTIMENT-TOPIC MODEL* BERBASIS *EXTENDED LONG SHORT-TERM MEMORY* UNTUK ANALISIS SENTIMEN

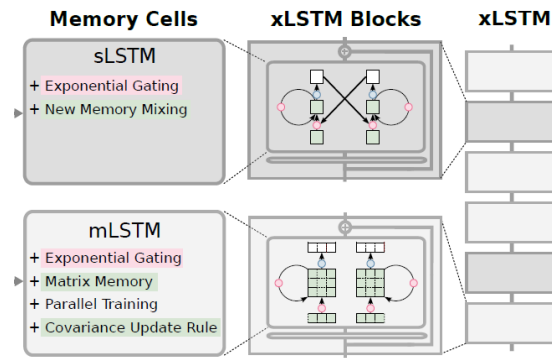
Bab ini menjelaskan arsitektur dan metode yang digunakan dalam pengembangan model JST berbasis xLSTM. Pembahasan dimulai dengan penjelasan mengenai xLSTM sebagai *shared encoder* yang menghasilkan representasi laten bersama, dilanjutkan dengan DEKM sebagai metode pendeteksian topik. Terakhir, dijelaskan arsitektur dan cara kerja model JST secara keseluruhan yang mengintegrasikan analisis sentimen dan pendeteksian topik dengan pendekatan MTL, sebagaimana diilustrasikan pada Gambar 3.1.



Gambar 3.1: Arsitektur model JST berbasis xLSTM

3.1 *Extended Long Short-Term Memory*

Extended Long Short-Term Memory (xLSTM) merupakan pengembangan dari arsitektur LSTM yang bertujuan meningkatkan kapasitas memori dan stabilitas pelatihan pada jaringan rekuren, sekaligus mempertahankan kemampuan LSTM dalam menangkap dependensi jangka panjang (Beck et al., 2024). xLSTM terdiri atas dua varian utama, yaitu *Scalar Long Short-Term Memory* (sLSTM) dan *Matrix Long Short-Term Memory* (mLSTM), yang masing-masing berperan sebagai komponen utama dalam pembentukan arsitektur xLSTM secara keseluruhan, sebagaimana ditunjukkan pada Gambar 3.2.



Gambar 3.2: Arsitektur xLSTM (Beck et al., 2024).

Peningkatan kinerja xLSTM dicapai melalui mekanisme *exponential gating*, yang memungkinkan pembaruan memori dilakukan secara lebih fleksibel. sLSTM memanfaatkan mekanisme ini bersama dengan *new memory mixing* untuk menggabungkan informasi lama dan baru dalam bentuk memori skalar. Sementara itu, mLSTM menggunakan *matrix memory* yang memungkinkan pemrosesan komputasi dilakukan secara paralel, sehingga lebih efisien untuk data berurutan yang panjang (Alharthi & Mahmood, 2024). Kedua varian ini kemudian diintegrasikan dalam satu *residual block backbone* untuk membentuk arsitektur xLSTM secara keseluruhan.

3.1.1 Scalar Long Short-Term Memory

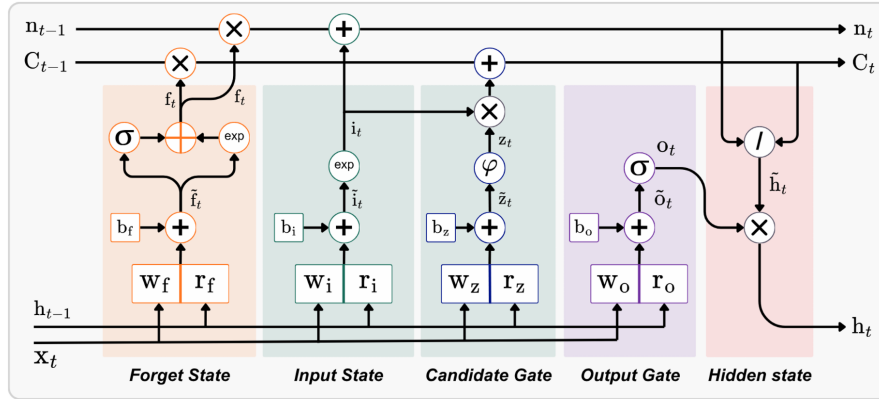
Scalar Long Short-Term Memory (sLSTM) merupakan salah satu varian dari xLSTM yang beroperasi pada tingkat skalar untuk setiap unit memorinya. sLSTM dikembangkan untuk meningkatkan stabilitas gradien serta efisiensi pembaruan memori pada jaringan rekuren (Beck et al., 2024). Salah satu komponen utama dalam sLSTM adalah mekanisme *exponential gating*, yaitu penggunaan fungsi eksponensial pada *gate* untuk mengontrol aliran informasi yang masuk ke dalam memori.

Pada LSTM, penggunaan fungsi sigmoid membatasi nilai *gate* dalam rentang $(0, 1)$, sehingga kemampuannya membedakan informasi yang sangat penting dari yang kurang penting menjadi terbatas. Melalui fungsi eksponensial, *gate* dapat memberikan bobot yang lebih besar pada informasi yang dianggap penting, sehingga model mampu mempertahankan informasi relevan dalam urutan yang panjang.

Selain itu, sLSTM memperkenalkan konsep *new memory mixing*, yaitu mekanisme yang menggabungkan informasi dari langkah sebelumnya dan informasi saat ini secara bersamaan saat memperbarui memori (Alharthi & Mahmood, 2024). Hal ini berbeda dengan LSTM yang hanya memanfaatkan *forget gate* untuk memutuskan berapa banyak memori lama yang dipertahankan.

3.1.1.1 Arsitektur sLSTM

sLSTM dirancang untuk menangkap hubungan antarelemen dalam data berurutan dengan lebih stabil melalui mekanisme pembaruan memori yang teratur. Pada arsitekturnya, sLSTM memperbarui memori skalar pada setiap langkah waktu t melalui mekanisme rekuren yang melibatkan *input gate*, *forget gate*, dan *output gate*, sebagaimana ditunjukkan pada Gambar 3.3.



Gambar 3.3: Arsitektur sLSTM (Alharthi & Mahmood, 2024), telah diolah kembali.

Proses komputasi pada sLSTM diawali dengan menghitung nilai *gate* dan kandidat memori berdasarkan *input* saat ini x_t serta *hidden state* sebelumnya h_{t-1} . Nilai-nilai tersebut kemudian digunakan untuk memperbarui *cell state* dan menghasilkan *hidden state* yang baru.

Pada seluruh persamaan berikut, W merepresentasikan matriks bobot, w merepresentasikan vektor bobot, dan b merupakan vektor bias. Kandidat memori z_t merepresentasikan informasi baru yang akan ditambahkan ke dalam memori. Nilai z_t diperoleh melalui fungsi aktivasi non-linear $\varphi(\cdot)$ pada kombinasi linear antara *input* x_t dan *hidden state* sebelumnya h_{t-1} , sebagaimana dirumuskan pada Persamaan (3.1):

$$z_t = \varphi(\tilde{z}_t), \quad \tilde{z}_t = w_z^\top x_t + r_z h_{t-1} + b_z. \quad (3.1)$$

Input gate i_t mengatur seberapa besar informasi baru z_t yang dimasukkan ke dalam memori. sLSTM menggunakan fungsi eksponensial sehingga model dapat memberikan bobot yang lebih besar pada informasi yang dianggap penting, sebagaimana dirumuskan pada Persamaan (3.2):

$$i_t = \exp(\tilde{i}_t), \quad \tilde{i}_t = w_i^\top x_t + r_i h_{t-1} + b_i. \quad (3.2)$$

Forget gate f_t mengatur seberapa besar informasi dari *cell state* sebelumnya yang dipertahankan. Nilai f_t dapat dihitung menggunakan fungsi sigmoid maupun fungsi eksponensial, sebagaimana ditunjukkan pada Persamaan (3.3):

$$f_t = \sigma(\tilde{f}_t) \quad \text{atau} \quad \exp(\tilde{f}_t), \quad \tilde{f}_t = \mathbf{w}_f^\top \mathbf{x}_t + r_f h_{t-1} + b_f. \quad (3.3)$$

Penggunaan fungsi eksponensial pada *gate* tertentu berpotensi menyebabkan ketidakstabilan numerik akibat nilai yang meningkat sangat signifikan. Maka, sLSTM menerapkan mekanisme stabilisasi melalui Persamaan (3.4)–(3.6) sebelum *gate* digunakan dalam pembaruan memori:

$$m_t = \max(\log(f_t) + m_{t-1}, \log(i_t)), \quad (3.4)$$

$$i'_t = \exp(\log(i_t) - m_t) = \exp(\tilde{i}_t - m_t), \quad (3.5)$$

$$f'_t = \exp(\log(f_t) + m_{t-1} - m_t). \quad (3.6)$$

Cell state c_t diperbarui menggunakan *gate* yang telah distabilkan, yaitu i'_t dan f'_t , melalui kombinasi antara memori dari langkah sebelumnya c_{t-1} dan kandidat memori baru z_t , sebagaimana dirumuskan pada Persamaan (3.7):

$$c_t = f'_t c_{t-1} + i'_t z_t. \quad (3.7)$$

Untuk menjaga kestabilan skala nilai *cell state* pada urutan yang panjang, sLSTM menggunakan variabel normalisasi n_t yang mengakumulasi pengaruh kumulatif *gate* sepanjang urutan, sebagaimana dirumuskan pada Persamaan (3.8):

$$n_t = f'_t n_{t-1} + i'_t. \quad (3.8)$$

Output gate o_t menentukan besarnya informasi memori yang diteruskan ke *output*, sebagaimana ditunjukkan pada Persamaan (3.9):

$$o_t = \sigma(\tilde{o}_t), \quad \tilde{o}_t = \mathbf{w}_o^\top \mathbf{x}_t + r_o h_{t-1} + b_o. \quad (3.9)$$

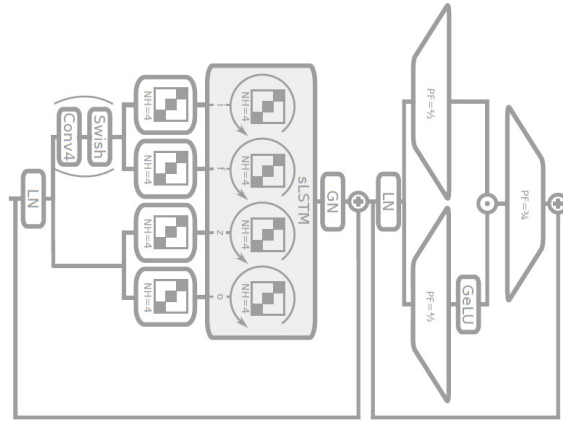
Hidden state h_t diperoleh dengan menormalkan *cell state* menggunakan n_t agar nilainya tetap dalam rentang yang terkendali, kemudian mengalikannya dengan *output gate* o_t ,

sebagaimana dirumuskan pada Persamaan (3.10):

$$h_t = o_t \tilde{h}_t, \quad \tilde{h}_t = \frac{c_t}{n_t}. \quad (3.10)$$

3.1.1.2 Blok sLSTM

Blok sLSTM mengintegrasikan sel sLSTM ke dalam struktur *residual block* untuk meningkatkan kapasitas representasi dan stabilitas pelatihan. Secara umum, blok ini memproses *input* secara bertahap mulai dari ekstraksi informasi kontekstual awal, pemrosesan rekuren oleh sel sLSTM, hingga penggabungan kembali dengan *input* melalui *skip connection* dan *Layer Normalization*. Struktur internal blok sLSTM ditunjukkan pada Gambar 3.4.



Gambar 3.4: Arsitektur blok sLSTM (Beck et al., 2024).

Sebelum memasuki sel sLSTM, *input* blok terlebih dahulu dilewatkan melalui lapisan *causal convolution* dengan fungsi aktivasi Swish untuk menangkap hubungan antar token yang berdekatan. Hasil konvolusi kemudian diproyeksikan menggunakan blok linier diagonal ke dalam beberapa bagian terpisah yang disebut *head*. Setiap *head* memproses bagiannya secara paralel dan hasilnya dinormalisasi menggunakan *Group Normalization*.

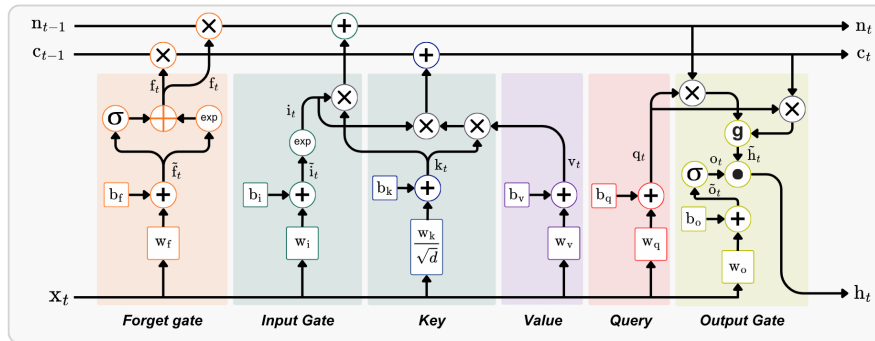
Representasi yang dihasilkan selanjutnya diteruskan ke sel sLSTM sebagai *input gate*, *forget gate*, *output gate*, dan pembaruan *cell state*. *Output* sel sLSTM kemudian diproyeksikan ke dimensi yang lebih tinggi melalui *gated MLP* dengan aktivasi GeLU, lalu dikembalikan ke dimensi awal melalui *down-projection*. Hasil akhir digabungkan dengan *input* blok melalui *skip connection* dan dinormalisasi menggunakan *Layer Normalization*.

3.1.2 Matrix Long Short-Term Memory

Matrix Long Short-Term Memory (mLSTM) merupakan varian dari arsitektur xLSTM yang menggunakan representasi memori berbentuk matriks berdimensi $\mathbb{R}^{d \times d}$, berbeda dengan LSTM yang menggunakan vektor. Dengan struktur matriks ini, model dapat menangkap hubungan antar dimensi fitur lebih mendalam karena setiap pasangan dimensi dapat berinteraksi langsung di dalam memori (Beck et al., 2024). Selain itu, karena mLSTM tidak memiliki koneksi rekuren pada *gate*-nya, pembaruan memori dapat dilakukan secara paralel sehingga proses pelatihan menjadi lebih efisien dibandingkan LSTM (Alharthi & Mahmood, 2024).

3.1.2.1 Arsitektur mLSTM

mLSTM menggunakan matriks memori dua dimensi $C_t \in \mathbb{R}^{d \times d}$ sebagai pengganti *cell state* berbentuk vektor pada LSTM konvensional, serta menghilangkan koneksi rekuren pada seluruh *gate* sehingga komputasi dapat dilakukan secara paralel selama pelatihan. Arsitektur mLSTM ditunjukkan pada Gambar 3.5.



Gambar 3.5: Arsitektur mLSTM (Alharthi & Mahmood, 2024), telah diolah kembali.

Pada seluruh persamaan berikut, W merepresentasikan matriks bobot, w merepresentasikan vektor bobot, dan b merupakan vektor bias. Komputasi mLSTM dimulai dengan memproyeksikan *input* x_t ke dalam tiga vektor representasi, yaitu *query* q_t , *key* k_t , dan *value* v_t , yang mengadaptasi prinsip serupa dengan Transformer, sebagaimana dirumuskan pada Persamaan (3.11):

$$q_t = W_q x_t + b_q, \quad k_t = \frac{1}{\sqrt{d}} W_k x_t + b_k, \quad v_t = W_v x_t + b_v. \quad (3.11)$$

Faktor $\frac{1}{\sqrt{d}}$ pada *key* berfungsi mencegah nilai *dot-product* yang terlalu besar, serupa dengan prinsip *scaled dot-product attention*. Secara definisi, *value* v_t adalah informasi

yang akan disimpan, *key* $\mathbf{k}_t$ menentukan letak informasi tersebut disimpan, dan *query* $\mathbf{q}_t$ menentukan informasi yang ingin diambil kembali dari memori.

Input gate i_t menentukan besarnya informasi baru yang dimasukkan ke dalam memori matriks. *Gate* ini hanya bergantung pada *input* saat ini $\mathbf{x}_t$ dan dihitung menggunakan fungsi eksponensial sebagaimana ditunjukkan pada Persamaan (3.12):

$$i_t = \exp(\tilde{i}_t), \quad \tilde{i}_t = \mathbf{w}_i^\top \mathbf{x}_t + b_i. \quad (3.12)$$

Forget gate f_t mengatur proporsi matriks memori lama yang dipertahankan dan dapat dihitung menggunakan fungsi sigmoid maupun fungsi eksponensial sebagaimana ditunjukkan pada Persamaan (3.13):

$$f_t = \sigma(\tilde{f}_t) \quad \text{atau} \quad \exp(\tilde{f}_t), \quad \tilde{f}_t = \mathbf{w}_f^\top \mathbf{x}_t + b_f. \quad (3.13)$$

Seperti pada sLSTM, penggunaan fungsi eksponensial pada *gate* tertentu berpotensi menimbulkan ketidakstabilan numerik. Oleh karena itu, mLSTM menerapkan mekanisme stabilisasi melalui Persamaan (3.14)–(3.16) sebelum *gate* digunakan dalam pembaruan memori:

$$m_t = \max(\log(f_t) + m_{t-1}, \log(i_t)), \quad (3.14)$$

$$i'_t = \exp(\log(i_t) - m_t) = \exp(\tilde{i}_t - m_t), \quad (3.15)$$

$$f'_t = \exp(\log(f_t) + m_{t-1} - m_t). \quad (3.16)$$

Matriks memori $\mathbf{C}_t$ diperbarui dengan *gate* yang telah distabilkan sebagaimana ditunjukkan pada Persamaan (3.17), dengan cara mempertahankan sebagian memori lama dan menambahkan informasi baru dalam bentuk perkalian luar (*outer product*) antara $\mathbf{v}_t$ dan $\mathbf{k}_t$:

$$\mathbf{C}_t = f'_t \mathbf{C}_{t-1} + i'_t \mathbf{v}_t \mathbf{k}_t^\top. \quad (3.17)$$

dengan $f'_t \mathbf{C}_{t-1}$ merupakan memori lama yang dipertahankan, sedangkan $i'_t \mathbf{v}_t \mathbf{k}_t^\top$ adalah informasi baru yang ditambahkan ke dalam memori.

Untuk menjaga kestabilan numerik selama proses pembaruan, mLSTM memperkenalkan variabel normalisasi $\mathbf{n}_t$ yang mengakumulasi pengaruh *gate* terhadap vektor *key* sepanjang urutan, sebagaimana dirumuskan pada Persamaan (3.18):

$$\mathbf{n}_t = f'_t \mathbf{n}_{t-1} + i'_t \mathbf{k}_t. \quad (3.18)$$

Output gate $\mathbf{o}_t$ mengatur aliran informasi dari memori ke *output* jaringan. Seperti *gate* lainnya pada mLSTM, *gate* ini juga tidak memiliki koneksi rekuren dan dihitung menggunakan Persamaan (3.19):

$$\mathbf{o}_t = \sigma(\tilde{\mathbf{o}}_t), \quad \tilde{\mathbf{o}}_t = \mathbf{W}_o \mathbf{x}_t + \mathbf{b}_o. \quad (3.19)$$

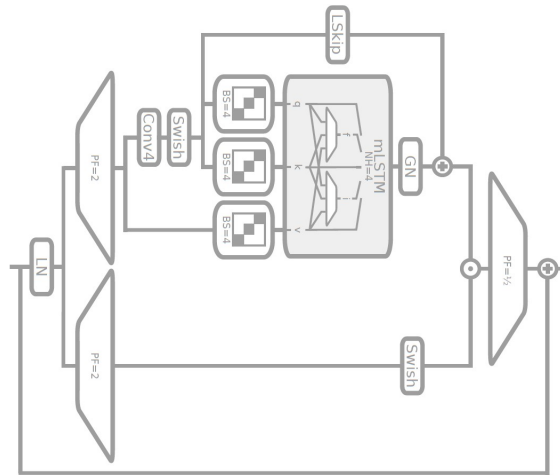
Hidden state $\mathbf{h}_t$ diperoleh dengan mengambil informasi dari matriks memori $\mathbf{C}_t$ menggunakan *query* $\mathbf{q}_t$, kemudian hasilnya dinormalisasi agar nilainya tetap dalam rentang yang terkendali, dan dikontrol oleh *output gate* $\mathbf{o}_t$, sebagaimana ditunjukkan pada Persamaan (3.20):

$$\mathbf{h}_t = \mathbf{o}_t \odot \tilde{\mathbf{h}}_t, \quad \tilde{\mathbf{h}}_t = \frac{\mathbf{C}_t \mathbf{q}_t}{\max(|\mathbf{n}_t^\top \mathbf{q}_t|, 1)}. \quad (3.20)$$

Pembagi $\max(|\mathbf{n}_t^\top \mathbf{q}_t|, 1)$ memastikan penyebut tidak mendekati nol, yaitu kondisi yang bisa terjadi ketika *dot-product* antara $\mathbf{n}_t$ dan $\mathbf{q}_t$ sangat kecil sehingga berpotensi menghasilkan nilai yang tidak stabil.

3.1.2.2 Blok mLSTM

Blok mLSTM mengintegrasikan sel mLSTM ke dalam struktur *residual block* dengan pendekatan *pre-up projection*. Pada pendekatan ini, *input* blok terlebih dahulu diproyeksikan ke dimensi yang lebih tinggi melalui dua jalur linier sebelum diproses lebih lanjut. Jalur pertama digunakan sebagai *input* utama bagi sel mLSTM, sedangkan jalur kedua menghasilkan *external output gate*. Struktur blok mLSTM secara keseluruhan ditunjukkan pada Gambar 3.6.



Gambar 3.6: Arsitektur Blok mLSTM (Beck et al., 2024).

Sebelum memasuki sel mLSTM, *input* dari jalur pertama dapat dilewatkan melalui *causal convolution* dengan ukuran *kernel* 4 untuk menangkap hubungan antar token yang berdekatan. *Input* kemudian diproyeksikan melalui proyeksi linier blok-diagonal ke dalam beberapa *head*. Setiap *head* menghasilkan vektor *query*, *key*, dan *value* yang diproses secara paralel dalam struktur *multi-head*. Seluruh *output head* kemudian digabungkan dan dinormalisasi menggunakan *Group Normalization*.

Hasil tersebut selanjutnya digabungkan dengan *skip connection* dari *input* awal yang dapat dipelajari, lalu dikontrol oleh *external output gate* melalui mekanisme *component-wise gating*. Pada tahap akhir, *output* diproyeksikan kembali ke dimensi semula melalui *down-projection* dan dinormalisasi menggunakan *Layer Normalization*. Dengan demikian, dimensi *output* blok tetap konsisten dengan dimensi *input* awal sehingga dapat diteruskan ke blok berikutnya melalui *residual connection*.

3.2 Deep Embedded K-Means

Deep Embedded K-Means (DEKM) merupakan metode *deep clustering* yang dikembangkan untuk mengatasi keterbatasan *clustering* konvensional pada data berdimensi tinggi (Guo et al., 2021). Berbeda dengan *K-Means* yang langsung melakukan pengelompokan pada fitur asli, DEKM memperluas mekanisme pengelompokan dengan terlebih dahulu mempelajari representasi laten yang lebih informatif melalui *autoencoder*. Dengan representasi laten tersebut, proses pengelompokan tidak lagi bergantung pada ruang fitur asli yang belum tentu mampu memisahkan *cluster* dengan baik.

Pada DEKM, *autoencoder* terdiri atas fungsi *encoder* $f(\cdot)$ dan *decoder* $g(\cdot)$. *Encoder* memetakan data $\mathbf{x} \in \mathbb{R}^{d \times 1}$ ke representasi laten $\mathbf{h} = f(\mathbf{x}) \in \mathbb{R}^{e \times 1}$, sedangkan *decoder*

merekonstruksi data melalui $\hat{x} = g(f(x))$. Pelatihan *autoencoder* dilakukan dengan meminimalkan *reconstruction loss* berikut:

$$\mathcal{L}_1 = \sum_{i=1}^n \|x_i - \hat{x}_i\|^2. \quad (3.21)$$

Setelah pelatihan selesai, *decoder* tidak lagi digunakan. Representasi laten $\mathbf{H} = f(\mathbf{X})$ yang dihasilkan *encoder* selanjutnya menjadi dasar proses pengelompokan, dengan fungsi objektif sebagai berikut:

$$\mathcal{L}_2 = \sum_{i=1}^K \sum_{h \in C_i} \|\mathbf{h} - \boldsymbol{\mu}_i\|^2, \quad (3.22)$$

dengan C_i menyatakan himpunan anggota *cluster* ke- i , $\boldsymbol{\mu}_i$ menyatakan *centroid*-nya, dan K menyatakan jumlah *cluster*.

Meskipun demikian, ruang laten yang dihasilkan *autoencoder* belum tentu memiliki struktur *cluster* yang terlihat jelas. Untuk mengatasinya, DEKM mentransformasi ruang laten $\mathbf{H}$ ke ruang baru $\mathbf{Y} = \mathbf{V}\mathbf{H}$ menggunakan matriks ortonormal $\mathbf{V}$, yaitu matriks yang kolom-kolomnya saling tegak lurus dan memiliki panjang satu (Anton, 2013). Transformasi ini merotasi ruang laten tanpa mengubah jarak antar titik data sehingga struktur geometri *cluster* tetap terjaga. Tujuannya adalah agar arah yang paling membedakan antar *cluster* menjadi lebih menonjol pada dimensi awal $\mathbf{Y}$, sedangkan dimensi akhir memuat informasi yang relatif kurang penting.

Matriks $\mathbf{V}$ diperoleh melalui dekomposisi *eigen* terhadap matriks sebaran dalam-*cluster* $\mathbf{S}_w$, yaitu matriks yang mengukur penyebaran data di dalam setiap *cluster*. *Eigenvalue* yang lebih kecil menunjukkan arah dengan penyebaran dalam-*cluster* yang lebih kecil sehingga memuat informasi *cluster* yang lebih kuat. Oleh karena itu, DEKM meminimalkan fungsi objektif $\mathcal{L}_3$ dengan solusi optimal berupa *eigenvector* yang memiliki *eigenvalue* terkecil dari $\mathbf{S}_w$:

$$\mathbf{S}_w = \sum_{i=1}^k \sum_{h \in C_i} (\mathbf{h} - \boldsymbol{\mu}_i)(\mathbf{h} - \boldsymbol{\mu}_i)^\top. \quad (3.23)$$

Transformasi dilakukan dengan meminimalkan sebaran dalam-*cluster* pada ruang baru:

$$\mathcal{L}_3 = \text{Trace}(\mathbf{V}\mathbf{S}_w\mathbf{V}^\top). \quad (3.24)$$

Nilai *eigen* yang lebih kecil menunjukkan arah di mana data antar *cluster* lebih rapat atau kurang tersebar, sehingga arah tersebut justru bisa digunakan untuk membedakan

cluster. Kolom-kolom V diurutkan berdasarkan nilai *eigen* dari yang terkecil ke terbesar, sehingga dimensi-dimensi awal pada Y merepresentasikan arah yang paling berguna untuk pengelompokan, sedangkan dimensi terakhir merepresentasikan arah yang paling sedikit memberikan informasi tentang struktur *cluster*.

Untuk meningkatkan kualitas representasi, DEKM memperbaiki dimensi terakhir yang paling tidak informatif tersebut. Untuk setiap $y \in Y$ yang termasuk dalam *cluster* C_i , dibentuk vektor baru y' yang identik dengan y kecuali pada dimensi terakhir, yang digantikan oleh komponen terakhir dari *centroid* hasil transformasi $m_i = V\mu_i$. Fungsi *loss* untuk optimasi ini dirumuskan sebagai:

$$\mathcal{L}_4 = \sum_{i=1}^k \sum_{y \in C_i} \|y - y'\|^2. \quad (3.25)$$

Optimasi dilakukan secara iteratif menggunakan pendekatan *mini-batch*. Setelah representasi diperbarui, pengelompokan dijalankan kembali untuk memperoleh pembagian *cluster* dan *centroid* yang baru. Proses ini diulang hingga fungsi *loss* konvergen atau struktur *cluster* yang dihasilkan sudah stabil.

3.3 Model Joint Sentiment–Topic Berbasis Extended Long Short–Term Memory

Model JST pada penelitian ini menggunakan pendekatan MTL dengan skema *hard parameter sharing*. xLSTM berperan sebagai *shared encoder* yang memproses dokumen ulasan menjadi representasi laten bersama z , kemudian digunakan secara bersamaan oleh dua jalur tugas, yaitu analisis sentimen dan pendeteksian topik. Arsitektur xLSTM terdiri atas dua jenis blok, yaitu blok mLSTM dan blok sLSTM, yang dapat dikombinasikan dalam berbagai konfigurasi. Konfigurasi xLSTM ditulis dalam bentuk xLSTM[$a:b$], dengan a menunjukkan jumlah blok mLSTM dan b menunjukkan jumlah blok sLSTM. Penelitian ini menguji empat konfigurasi, yaitu:

1. xLSTM[1:0] (satu blok mLSTM),
2. xLSTM[0:1] (satu blok sLSTM),
3. xLSTM[1:1] (kombinasi satu blok sLSTM dan blok mLSTM),
4. xLSTM[7:1] (tujuh blok mLSTM).

Selain keempat konfigurasi xLSTM tersebut, penelitian ini juga menyertakan LSTM sebagai model pembanding. LSTM digunakan sebagai *encoder* dalam arsitektur JST yang

sama, sehingga perbandingan yang dilakukan membedakan jenis *encoder* yang digunakan sementara komponen lainnya tetap sama. Perbandingan ini bertujuan untuk mengukur sejauh mana penggunaan xLSTM memberikan peningkatan performa dibandingkan LSTM dalam model JST.

3.3.1 Pra-pelatihan *Encoder Extended Long Short-Term Memory*

Sebelum digunakan pada tahap pelatihan utama, xLSTM terlebih dahulu melalui tahap pelatihan awal (*pre-training*). Pendekatan ini didasarkan pada penelitian Guo et al. (2021) yang menunjukkan bahwa *encoder* yang dilatih sebelum proses *clustering* cenderung menghasilkan representasi dokumen yang lebih terstruktur dan bermakna. Pada tahap ini, xLSTM dilatih menggunakan fungsi *loss KL Divergence* $\mathcal{L}_{KL}$ yang telah didefinisikan pada Persamaan 2.16. xLSTM belajar menghasilkan distribusi probabilitas dari kata-kata penting yang mencerminkan kandungan polaritas sentimen dan topik dokumen.

Distribusi hasil prediksi tersebut kemudian dibandingkan dengan frekuensi relatif kemunculan kata-kata penting pada dokumen aktual. xLSTM diharapkan dapat mempelajari kata-kata yang paling merepresentasikan isi dokumen dan menghasilkan representasi laten yang lebih informatif. Bobot *encoder* yang diperoleh dari tahap pra-pelatihan selanjutnya digunakan sebagai inisialisasi parameter pada tahap *joint training* model JST.

3.3.2 Pembentukan Representasi Laten

Pada tahap *joint training*, setiap dokumen terlebih dahulu ditokenisasi dengan panjang maksimum 128 token. Setiap token kemudian direpresentasikan menggunakan *embedding* GloVe berdimensi 100, lalu diproyeksikan ke dimensi 256 melalui lapisan linier dan *Layer Normalization*. Representasi token tersebut selanjutnya diproses oleh xLSTM dua arah, yaitu arah maju (*forward*) untuk menangkap konteks kata-kata sebelumnya dan arah mundur (*backward*) untuk menangkap konteks kata-kata sesudahnya. Hasil dari kedua arah digabungkan dan diproyeksikan kembali menjadi satu vektor berdimensi 256.

Untuk memperoleh satu representasi per dokumen, *output* xLSTM diagregasi menggunakan *masked mean pooling*, yaitu dengan menghitung rata-rata representasi token hanya pada posisi token yang valid dan mengabaikan token *padding*. Representasi

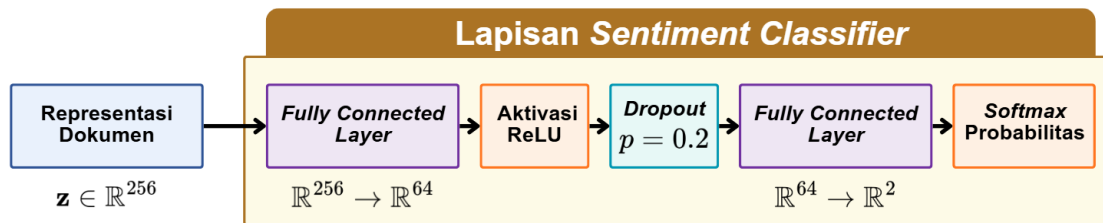
dokumen $z \in \mathbb{R}^{256}$ dihitung sebagai:

$$z = \frac{\sum_{t=1}^L m_t \mathbf{h}_t}{\sum_{t=1}^L m_t}, \quad (3.26)$$

dengan $\mathbf{h}_t$ adalah representasi token pada posisi ke- t , m_t adalah nilai *attention mask* yang bernilai 1 untuk token valid dan 0 untuk token *padding*, serta L adalah panjang urutan token. Vektor z selanjutnya digunakan bersama oleh jalur analisis sentimen dan jalur pendeteksian topik.

3.3.3 Jalur Analisis Sentimen

Pada jalur analisis sentimen, vektor z dilewatkan ke *sentiment head* yang terdiri atas lapisan *fully connected* $\mathbb{R}^{256} \rightarrow \mathbb{R}^{64}$ dengan aktivasi ReLU, *dropout* dengan probabilitas $p = 0,2$ sebagai regularisasi, dan lapisan *output* $\mathbb{R}^{64} \rightarrow \mathbb{R}^2$ dengan fungsi *softmax* untuk menghasilkan distribusi probabilitas kelas positif dan negatif. Dimensi 256 tetap dipertahankan pada jalur ini agar seluruh kapasitas representasi kontekstual xLSTM dapat dimanfaatkan untuk membedakan variasi sentimen. Fungsi *loss* yang digunakan adalah BCE dengan *label smoothing* untuk mencegah *overconfidence* model terhadap label pelatihan. Alur lengkap jalur analisis sentimen ditunjukkan pada Gambar 3.7.

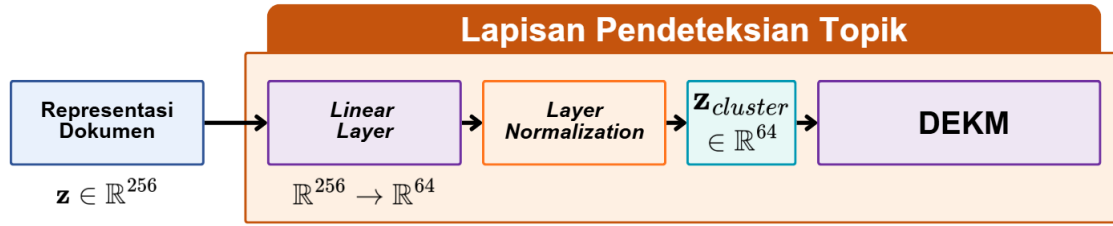


Gambar 3.7: Alur lengkap jalur analisis sentimen

3.3.4 Jalur Pendeteksian Topik

Pada jalur pendeteksian topik, vektor z terlebih dahulu dikompresi melalui *projection head* berupa lapisan *linear* $\mathbb{R}^{256} \rightarrow \mathbb{R}^{64}$ yang diikuti *layer normalization*, menghasilkan $z_{\text{cluster}} \in \mathbb{R}^{64}$. Kompresi ini dilakukan karena algoritma *clustering* berbasis jarak rentan terhadap *curse of dimensionality* pada ruang berdimensi tinggi (Keogh & Mueen, 2010), sehingga proyeksi ke dimensi yang lebih rendah membantu dokumen yang serupa berada lebih berdekatan dalam ruang representasi. Vektor z_{cluster} selanjutnya digunakan sebagai *input* bagi komponen DEKM sebagaimana dijelaskan pada Subbab 3.2 untuk melakukan

pengelompokan dokumen berdasarkan topik. Alur lengkap jalur pendeteksian topik ditunjukkan pada Gambar 3.8.



Gambar 3.8: Alur lengkap jalur pendeteksian topik

3.3.5 Joint Loss dan Joint Training

Setelah representasi dokumen z diproses oleh jalur analisis sentimen dan jalur pendeteksian topik, masing-masing jalur menghasilkan nilai *loss* yang kemudian digabungkan ke dalam satu fungsi objektif bersama (*joint loss*). Fungsi *joint loss* pada penelitian ini terdiri dari *loss* analisis sentimen dan *clustering loss* topik dari DEKM. Fungsi *loss* analisis sentimen $\mathcal{L}_{\text{BCE}}$ mengacu pada Persamaan (2.14), sedangkan fungsi *loss* DEKM $\mathcal{L}_{\text{DEKM}}$ mengacu pada Persamaan (3.25).

Proses *joint training* dibagi menjadi dua fase berdasarkan fungsi objektif yang digunakan, yaitu fase *warmup* dan fase pelatihan penuh. Pada fase *warmup*, optimasi hanya dilakukan menggunakan *loss* sentimen agar xLSTM terlebih dahulu membentuk representasi yang stabil sebelum komponen *clustering* diaktifkan. Fungsi objektif pada fase ini dirumuskan sebagai

$$\mathcal{L}_{\text{warmup}} = \eta \mathcal{L}_{\text{BCE}}, \quad (3.27)$$

dengan $\eta = 1,0$ sebagai bobot *loss* sentimen.

Setelah fase *warmup* selesai, pelatihan dilanjutkan ke fase pelatihan penuh di mana *loss* sentimen dan *clustering loss* digabungkan ke dalam fungsi objektif bersama yang dirumuskan sebagai

$$\mathcal{L}_{\text{joint}} = \eta \mathcal{L}_{\text{BCE}} + \gamma \mathcal{L}_{\text{DEKM}}, \quad (3.28)$$

dengan $\eta = 1,0$ sebagai bobot *loss* sentimen dan $\gamma = 1,0$ sebagai bobot *loss clustering*. Penetapan bobot yang sama bertujuan menjaga keseimbangan kontribusi kedua tugas selama proses pelatihan.

BAB 4

HASIL SIMULASI DAN PEMBAHASAN

Bab ini berisikan hasil simulasi dan analisis kinerja model JST berbasis xLSTM yang diusulkan. Data yang digunakan untuk menghasilkan performa model menggunakan *dataset* IMDB dan Yelp yang telah melalui tahapan pra-pemrosesan dan pembagian data. Kinerja model dianalisis dari dua aspek utama, yaitu analisis sentimen dan pendeteksian topik. Pada tugas analisis sentimen, performa model dievaluasi menggunakan metrik *accuracy*, *precision*, *recall*, dan *F1-score*. Sementara itu, pada pendeteksian topik, kualitas hasil *cluster* dianalisis menggunakan metrik skor *Topic Coherence–W2V* dan *Topic Diversity*. Hasil evaluasi ini digunakan untuk memberikan gambaran mengenai kemampuan model dalam menangkap keterkaitan antara sentimen dan topik secara terpadu.

4.1 Spesifikasi Mesin dan Perangkat Lunak

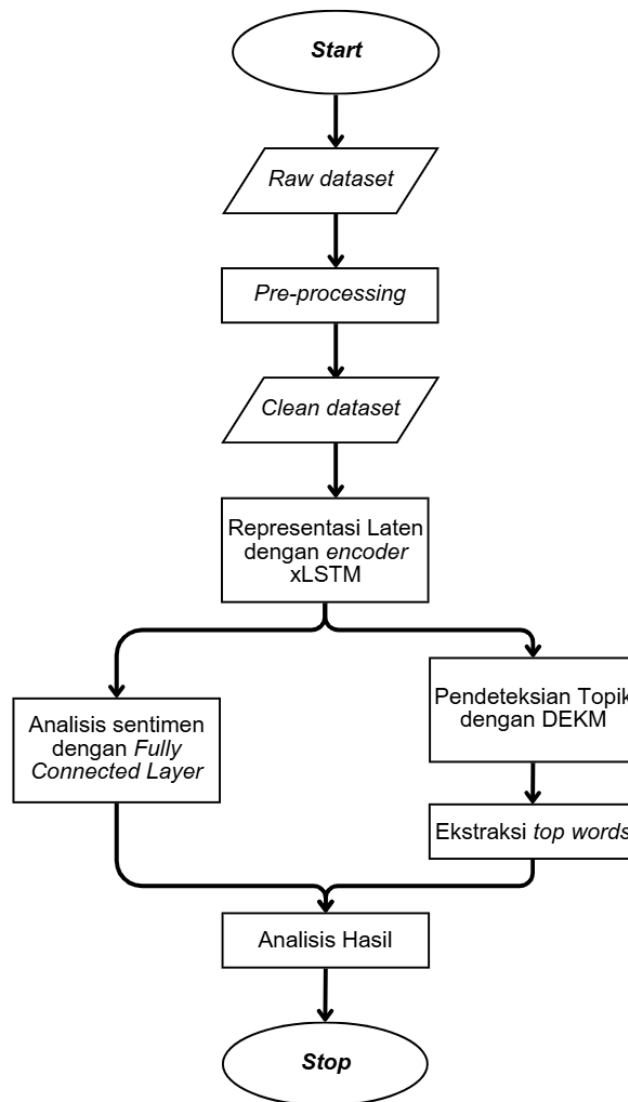
Spesifikasi mesin dan perangkat lunak yang digunakan dalam proses simulasi pada penelitian ini dapat dilihat pada Tabel 4.1.

Tabel 4.1: Spesifikasi mesin dan perangkat lunak

Mesin			
Central Processing Unit (CPU)			Intel(R) Core(TM) Ultra 7 155H
Memory			32 GB
Graphic Processing Unit (GPU)			Intel(R) Arc(TM) Graphics NVIDIA GeForce RTX 4050 Laptop GPU
Perangkat Lunak			
Perangkat Lunak Pemrograman			Google Colaboratory Visual Studio Code
Bahasa Pemrograman			Python 3.13.3
Pustaka yang Digunakan			- os, math, random, json, re - numpy - PyTorch (torch) - gensim, GloVe
			- transformers - datasets - tqdm - xlstm

4.2 Tahapan Umum Simulasi

Tahapan umum simulasi penelitian ini digambarkan dalam diagram alur pada Gambar 4.1.



Gambar 4.1: Diagram alur penelitian.

Proses dimulai dengan pengumpulan data ulasan yang selanjutnya melewati tahap pra-pemrosesan untuk menghasilkan data bersih yang akan digunakan dalam pemodelan. Data yang telah bersih kemudian menjadi *input* bagi xLSTM untuk menghasilkan representasi laten dokumen. Representasi dokumen tersebut selanjutnya digunakan secara bersamaan oleh dua jalur tugas utama, yaitu analisis sentimen dan pemodelan topik.

Pada jalur analisis sentimen, representasi laten diproses melalui *fully connected layer* untuk menghasilkan prediksi kelas sentimen. Pada jalur pendeteksian topik, representasi laten yang sama digunakan sebagai *input* bagi DEKM. Setelah itu, hasil akhir dari klasifikasi sentimen dan pendeteksian topik dinilai dengan metrik evaluasi yang sesuai.

4.3 Data

Subbab ini menjelaskan data yang digunakan dalam penelitian, meliputi sumber data, karakteristik data, serta tahapan awal persiapan data sebelum digunakan dalam implementasi model JST. Secara umum, proses pengolahan data pada penelitian ini diawali dengan pengumpulan *dataset* ulasan, dilanjutkan dengan tahap prapengolahan teks, pembagian data, serta pembentukan representasi teks sebagai *input* model.

4.3.1 Pengumpulan Data

Penelitian ini menggunakan dua *dataset* ulasan berbahasa inggris, yaitu IMDB Movie Reviews dan Yelp Review Dataset. Kedua *dataset* tersebut dipilih karena umum digunakan sebagai *benchmark* dalam penelitian analisis sentimen dan pendeteksian topik. Selain itu, kedua *dataset* tersebut dilengkapi dengan label sentimen biner.

Dataset IMDB Movie Reviews terdiri dari 30.000 ulasan film yang diunggah oleh (Maas et al., 2011) dan tersedia secara publik melalui platform Hugging Face dalam format *Apache Arrow*. *Dataset* IMDB memuat label sentimen positif dan negatif yang relatif seimbang, sehingga sesuai digunakan untuk tugas klasifikasi biner.

Yelp Review Dataset terdiri dari 50.000 ulasan pengguna terhadap berbagai layanan dan bisnis, seperti restoran dan toko. *Dataset* ini tersedia juga secara publik melalui platform Hugging Face yang diunggah oleh (Zhang et al., 2015) dalam format *Apache Arrow* dengan label sentimen positif dan negatif yang seimbang. Dalam penelitian ini, jumlah data Yelp yang digunakan disesuaikan agar proporsional dengan jumlah data IMDB, yaitu 30.000.

Sebagai contoh, beberapa cuplikan dokumen ulasan dari masing-masing dataset disajikan pada Tabel 4.2 dan Tabel 4.3. Kedua dataset dilengkapi label sentimen biner, dengan nilai 1 menunjukkan sentimen positif dan nilai 0 menunjukkan sentimen negatif. Cuplikan data ulasan IMDB ditampilkan pada Tabel 4.2 berikut:

Tabel 4.2: Cuplikan data ulasan IMDB (Maas et al., 2011).

No	Text	Label
1	If you like original gut-wrenching laughter, you will like this movie. If you are young or old, then you will love this movie; even my mom liked it. Great camp!	1

No	Text	Label
2	The plot is about the death of little children. Hopper is the one who has to investigate the killings ... Just don't watch the movie.	0
⋮	⋮	⋮
30.000	This cute animated short features two comic icons, Betty Boop and Henry. Henry is the bald, slightly portly boy from the comics who never speaks ... but everything works out in the end.	1

Selanjutnya, cuplikan data ulasan Yelp disajikan pada Tabel 4.3 berikut:

Tabel 4.3: Cuplikan data ulasan Yelp (Zhang et al., 2015).

No	Text	Label
1	This is by far the best dentist I have ever been to. She is honest and never tries to sell you a bunch of stuff that you don't need. We are very grateful ... worth the drive from Florida.	1
2	Far away from real Chinese food. Doesn't even taste as good as American style Chinese food.	0
⋮	⋮	⋮
30.000	Interactive video games that don't work well and some surround screen video that wasn't well done. Not worth the time and definitely not the price.	0

4.3.2 Prapengolahan Data

Tahap prapengolahan data dilakukan untuk membersihkan teks agar dapat digunakan secara optimal dalam proses pembentukan representasi dengan xLSTM. Karena xLSTM merupakan model sekuensial yang dirancang untuk menangkap hubungan kontekstual antartoken secara berurutan, penelitian ini menerapkan prapengolahan yang minimalis. Tahapan seperti *stemming*, penghapusan *stop words*, dan normalisasi tanda baca tidak

diterapkan, karena kata-kata fungsional seperti *stop words* masih mengandung informasi yang relevan bagi model dalam membedakan sentimen maupun topik ulasan.

Pada tahap selanjutnya, teks akan diproses menggunakan tokenisasi berbasis *WordPiece* yang mampu menangani kata-kata di luar kosakata dengan memecahnya menjadi unit *subword*, sehingga tidak diperlukan normalisasi kata secara manual. Oleh karena itu, prapengolahan pada penelitian ini difokuskan pada pembersihan elemen-elemen yang tidak memiliki makna bahasa, tanpa menghilangkan kata-kata yang masih mengandung informasi kontekstual. Adapun tahapan prapengolahan yang dilakukan adalah sebagai berikut:

1. *Casefolding*, yaitu mengubah seluruh teks menjadi huruf kecil untuk menghindari perbedaan representasi kata akibat variasi kapitalisasi.
2. Penghapusan tag HTML seperti `<br />`, `<b>`, dan elemen HTML lain yang tidak memiliki kandungan bahasa.
3. Penghapusan URL, yaitu menghapus tautan atau alamat situs web dengan pola `http://...` atau `www....` karena tidak mengandung informasi sentimen maupun topik.
4. Penghapusan karakter non-ASCII, yaitu menghapus emoji, simbol *unicode*, dan karakter khusus yang tidak termasuk dalam karakter teks standar yang berpotensi mengganggu proses tokenisasi.
5. Normalisasi *whitespace*, yaitu menggabungkan spasi berlebih, karakter *tab*, dan baris baru menjadi satu spasi tunggal agar teks memiliki format yang konsisten.

Hasil dari tahap prapengolahan ini berupa teks yang telah dibersihkan dan distandarisasi, tetapi tetap mempertahankan sebagian besar struktur kalimat asli. Teks yang telah melalui tahap ini selanjutnya digunakan pada proses tokenisasi dan pembentukan representasi teks menggunakan *encoder* xLSTM. Contoh perbandingan data teks sebelum dan sesudah melalui tahap prapengolahan dapat dilihat pada Tabel 4.4.

Tabel 4.4: Contoh hasil prapengolahan pada dataset IMDB dan Yelp

IMDB	
Sebelum Prapengolahan	Sesudah Prapengolahan
If you like original gut-wrenching laughter, you will like this movie. If you are young or old, then you will love this movie; even my mom liked it. Great camp!	if you like original gut-wrenching laughter, you will like this movie. if you are young or old, then you will love this movie; even my mom liked it. great camp!
The plot is about the death of little children. Hopper is the one who has to investigate the killings. During the movie, it appears that he has some troubles with his daughter.	the plot is about the death of little children. hopper is the one who has to investigate the killings. during the movie, it appears that he has some troubles with his daughter.
Yelp	
Sebelum Prapengolahan	Sesudah Prapengolahan
This is by far the best dentist I have ever been to. She is honest and never tries to sell you a bunch of stuff that you don't need.	this is by far the best dentist i have ever been to. she is honest and never tries to sell you a bunch of stuff that you don't need.
Interactive video games that don't work well & some surround screen video that wasn't well done. http://yelp.com/review Not worth the time.	interactive video games that don't work well some surround screen video that wasn't well done. not worth the time.

Hasil prapengolahan pada Tabel 4.4 menunjukkan bahwa perubahan utama yang dilakukan adalah pembersihan elemen-elemen non-linguistik dan penyeragaman format teks, bukan penyederhanaan isi kalimat. Dengan demikian, informasi kontekstual dalam ulasan tetap dipertahankan sehingga dapat dimanfaatkan lebih lanjut pada tahap tokenisasi dan pembentukan *embedding*.

4.3.3 Representasi Teks

Setelah melalui tahap prapengolahan data, teks selanjutnya diubah menjadi representasi numerik melalui proses tokenisasi dan pembentukan *embedding*. Tahap ini bertujuan untuk menghasilkan representasi yang dapat diproses oleh model xLSTM. Kualitas representasi teks yang dihasilkan akan memengaruhi kemampuan model dalam menangkap pola sentimen dan topik pada data ulasan.

Tokenisasi dilakukan menggunakan skema *WordPiece* yang mampu memecah kata menjadi unit *subword*, sehingga dapat menangani kata di luar kosakata dengan lebih baik. Setiap teks yang telah diprapengolahan dikonversi menjadi urutan *token ID* dengan panjang maksimum 128 token. Apabila panjang teks melebihi batas tersebut, maka dilakukan *truncation* atau pemotongan, sedangkan teks yang lebih pendek akan ditambahkan *padding*, yaitu menambahkan token 0 agar memiliki panjang yang seragam. Contoh hasil tokenisasi dapat dilihat pada Tabel 4.5.

Tabel 4.5: Contoh proses tokenisasi

Teks (hasil prapengolahan)	Hasil tokenisasi (token ID)
like original gut wrenching laughter like movie young old love movie	[101, 2066, 2434, 9535, 16258, 8452, 2066, 3185, 2402, 2214, 2293, 3185, 102, 0, ..., 0]

Hasil tokenisasi berupa token ID selanjutnya dipetakan menjadi representasi vektor melalui proses *embedding*. Pada penelitian ini, *embedding* yang digunakan adalah GloVe berdimensi 100 yang telah dilatih sebelumnya pada korpus besar. Setiap token dalam kosakata dipetakan ke dalam vektor *embedding* GloVe yang bersesuaian. Untuk token yang tidak terdapat dalam kamus GloVe, *embedding* diinisialisasi secara acak menggunakan distribusi normal agar tetap dapat dipelajari selama proses pelatihan. Contoh ilustrasi perubahan representasi dari token ID menjadi *embedding* dapat dilihat pada Tabel 4.6.

Tabel 4.6: Contoh proses *embedding*

Token ID	<i>Embedding</i>
[2066, 2434, 9535, ...]	[[0.12, -0.45, ..., 0.03], [0.08, 0.22, ..., -0.11], ...]

4.3.4 Pembagian Data

Setelah melalui tahap prapengolahan, data ulasan dari *dataset* IMDB dan Yelp dibagi ke dalam dua bagian, yaitu data latih (*training set*) dan data uji (*test set*). Pembagian data ini merujuk pada penelitian yang dilakukan oleh (Gowandi et al., 2021) dengan proporsi 80% untuk data latih dan 20% untuk data uji. Skema pembagian ini digunakan untuk memastikan bahwa proses pelatihan dan evaluasi kinerja model dapat dilakukan secara terpisah.

Data latih digunakan sebagai *input* utama dalam proses pelatihan parameter model, baik pada tugas klasifikasi sentimen maupun pendeteksian topik. Selanjutnya, data uji digunakan untuk mengevaluasi performa model secara menyeluruh terhadap data yang belum pernah dilihat sebelumnya.

4.4 Implementasi *Joint Sentiment–Topic Model*

Implementasi model dilakukan melalui tiga tahapan utama, yaitu perancangan arsitektur xLSTM, tahap pra-pelatihan untuk memperoleh representasi awal yang informatif, serta tahap *joint training* yang menggabungkan pelatihan sentimen dan topik dalam satu kerangka pelatihan. Konfigurasi pada masing-masing tahapan dijelaskan pada subbagian berikutnya. Seluruh implementasi dilakukan menggunakan bahasa pemrograman *Python* dengan *library* yang tercantum pada Tabel 4.1.

4.4.1 Konfigurasi *Encoder Extended Long Short–Term Memory*

Encoder pada penelitian ini menggunakan xLSTM dengan kombinasi blok mLSTM dan sLSTM. Hasil *word embedding* yang telah diperoleh dari tahap sebelumnya diproyeksikan ke dimensi 256 dan diproses melalui beberapa blok xLSTM sesuai konfigurasi yang diuji. *Output* dari arah maju (*forward*) dan mundur (*backward*) kemudian digabungkan melalui *projection layer* dan diringkas menggunakan *masked*

mean pooling untuk menghasilkan representasi dokumen. Konfigurasi parameter utama yang digunakan pada xLSTM disajikan pada Tabel 4.7.

Tabel 4.7: Konfigurasi utama *encoder* xLSTM

Komponen	Konfigurasi
Dimensi <i>embedding</i> GloVe	100
Panjang maksimum token	128
Dimensi <i>hidden state</i> xLSTM	256
Konfigurasi blok xLSTM	[1:0], [0:1], [1:1], [7:1]
Jumlah <i>head</i>	4
Ukuran <i>kernel</i>	4
Metode agregasi representasi	<i>Masked mean pooling</i>

4.4.2 Konfigurasi Pra-pelatihan xLSTM

Pra-pelatihan pada penelitian ini dilakukan sebelum tahap *joint training* untuk membentuk representasi awal pada xLSTM. Tahap ini menggunakan rekonstruksi distribusi kata kunci. Konfigurasi parameter utama yang digunakan pada tahap pra-pelatihan disajikan pada Tabel 4.8.

Tabel 4.8: Konfigurasi utama pra-pelatihan xLSTM

Parameter	Nilai
Panjang maksimum token	128
Dimensi <i>embedding</i> GloVe	100
Dimensi <i>hidden state</i> xLSTM	256
Fungsi <i>reconstruction loss</i>	KL <i>divergence</i>
<i>Batch Size</i>	96
<i>Learning Rate</i>	3×10^{-4}
Jumlah <i>Epoch</i>	12
<i>Optimizer</i>	AdamW
<i>Weight decay</i>	0.01

4.4.3 Konfigurasi *Joint Training*

Pada tahap *joint training*, konfigurasi pelatihan dibuat sama untuk setiap kombinasi xLSTM dan jumlah topik yang diuji. Tahap ini menggunakan xLSTM hasil pra-pelatihan sebagai pembentuk representasi bersama. Proses pelatihan dilakukan dengan menggabungkan *loss* klasifikasi sentimen dan *loss clustering*, dengan bobot masing-masing ditetapkan sebesar $\eta = 1,0$ untuk *loss* sentimen dan $\gamma = 1,0$ untuk *loss clustering*. Pelatihan dilakukan hingga maksimum 60 *epoch* dengan fase *warmup* selama

8 *epoch*. Pada fase *warmup*, model terlebih dahulu dilatih menggunakan *loss* sentimen sebelum komponen *clustering* diaktifkan.

Pada pendeteksian topik, ditetapkan tiga variasi jumlah topik, yaitu $K = 30, 50$, dan 80 , mengacu pada penelitian Gui et al. (2022). Setiap kombinasi konfigurasi xLSTM dan jumlah topik dijalankan sebanyak 3 *run* dengan *seed* berbeda, yaitu 42, 123, dan 456, untuk mengurangi pengaruh inisialisasi acak terhadap hasil. Parameter utama yang digunakan pada tahap *joint training* disajikan pada Tabel 4.9.

Tabel 4.9: Konfigurasi utama *joint training* model JST

Parameter	Nilai
Dimensi <i>projection head</i> topik	64
<i>Batch size</i>	256
<i>Learning rate</i>	2×10^{-4}
<i>Weight decay</i>	0.01
<i>Optimizer</i>	AdamW
Jumlah <i>epoch</i>	60
Jumlah <i>warmup epoch</i>	8
Jumlah topik (K)	30, 50, 80
Jumlah <i>run</i>	3
<i>Seed</i>	42, 123, 456
Bobot <i>loss</i> sentimen (η)	1.0
Bobot <i>loss clustering</i> (γ)	1.0

4.5 Analisis Kinerja Model *Joint Sentiment–Topic*

Bagian ini membahas kinerja model JST pada tiga aspek utama, yaitu kualitas representasi hasil pra-pelatihan, kinerja klasifikasi sentimen, dan kemampuan pendeteksian topik pada dokumen ulasan. Evaluasi dilakukan untuk mengukur sejauh mana representasi yang dihasilkan oleh xLSTM mampu mendukung kedua tugas tersebut secara efektif, dengan membandingkan hasil pada berbagai konfigurasi model yang telah diuji.

4.5.1 Analisis Kinerja Hasil Pra-pelatihan

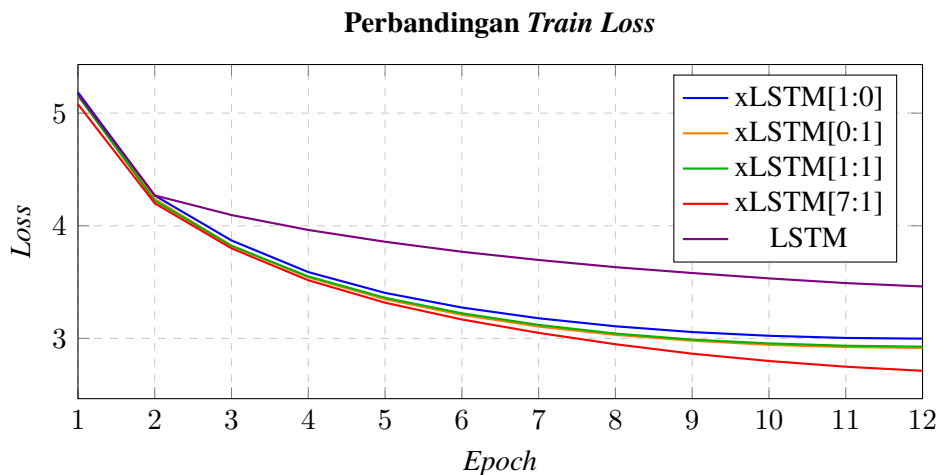
Kinerja pra-pelatihan dievaluasi menggunakan nilai *loss* pada data latih dan data validasi. Data yang digunakan pada tahap pra-pelatihan dibagi menjadi dua bagian, yaitu 98% sebagai data latih dan 2% sebagai data validasi. Pembagian ini dilakukan untuk meninjau apakah xLSTM mampu mempelajari representasi yang umum dan tidak hanya menghafalkan pola pada data latih. Nilai *loss* yang digunakan adalah *KL Divergence*

sebagaimana didefinisikan pada Persamaan 2.16. Hasil perbandingan *loss* pra-pelatihan ditunjukkan pada Tabel 4.10.

Tabel 4.10: Perbandingan *train loss* dan *validation loss* pra-pelatihan

Epoch	xLSTM[1:0]		xLSTM[0:1]		xLSTM[1:1]		xLSTM[7:1]		LSTM	
	train	val	train	val	train	val	train	val	train	val
1	5.1842	4.5357	5.1545	4.5109	5.1569	4.4761	5.0788	4.4278	5.1677	4.3716
2	4.2683	4.0576	4.2372	4.0167	4.2212	4.0255	4.1989	3.9702	4.2695	4.1785
3	3.8695	3.7333	3.8237	3.6897	3.8216	3.7017	3.7997	3.6606	4.0953	4.0497
4	3.5886	3.5265	3.5428	3.4783	3.5501	3.5023	3.5163	3.4661	3.9628	3.9559
5	3.4042	3.4099	3.3481	3.3451	3.3609	3.3771	3.3169	3.3470	3.8580	3.8939
6	3.2742	3.3210	3.2095	3.2529	3.2232	3.2731	3.1681	3.2467	3.7692	3.8386
7	3.1788	3.2689	3.1049	3.1972	3.1201	3.2133	3.0490	3.1738	3.6964	3.8128
8	3.1078	3.2239	3.0313	3.1509	3.0433	3.1686	2.9483	3.1195	3.6325	3.7733
9	3.0564	3.1985	2.9797	3.1263	2.9897	3.1452	2.8644	3.0739	3.5807	3.7437
10	3.0234	3.1848	2.9430	3.1081	2.9557	3.1291	2.7995	3.0382	3.5330	3.7245
11	3.0043	3.1815	2.9231	3.1110	2.9351	3.1254	2.7488	3.0095	3.4913	3.7092
12	2.9978	3.1753	2.9155	3.1040	2.9272	3.1173	2.7124	3.0037	3.4611	3.6983
Total waktu (menit)	34.7		367.2		768.1		4251.0		28.0	

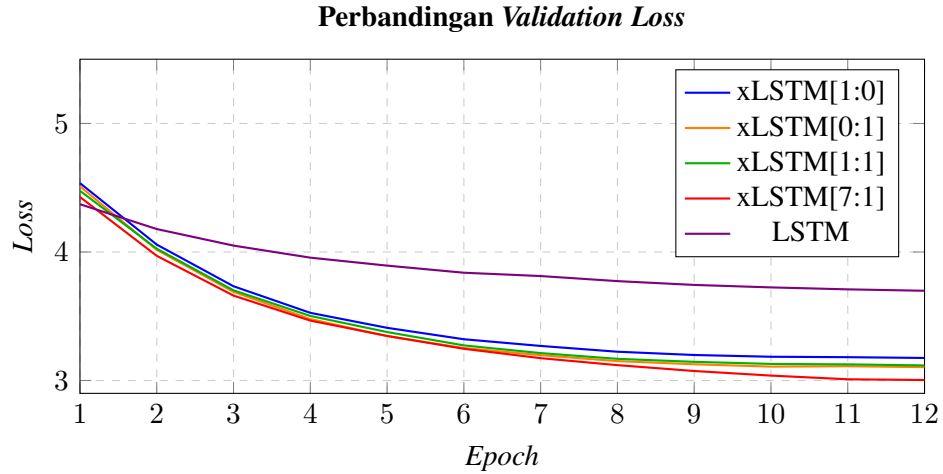
Berdasarkan Tabel 4.10, seluruh konfigurasi menunjukkan tren penurunan *train loss* yang konsisten seiring bertambahnya *epoch*. Di antara seluruh konfigurasi yang diuji, xLSTM[7:1] secara konsisten menghasilkan *train loss* terendah dengan nilai akhir sebesar 2.7124 pada *epoch* ke-12. Tren ini juga dapat diamati pada Gambar 4.2 berikut.



Gambar 4.2: Perbandingan *train loss* pra-pelatihan seluruh *encoder*.

Pola serupa juga terlihat pada *validation loss* yang ditunjukkan pada Gambar 4.3. xLSTM[7:1] mencatat nilai *validation loss* akhir terendah sebesar 3.0037, sementara xLSTM[1:0], xLSTM[0:1], dan xLSTM[1:1] menunjukkan performa yang relatif sebanding dengan nilai akhir masing-masing sebesar 3.1753, 3.1040, dan 3.1173.

Encoder LSTM mencatat nilai tertinggi sebesar 3.6983, menandakan keterbatasannya dalam mempelajari distribusi kata kunci dokumen dibandingkan seluruh konfigurasi xLSTM.



Gambar 4.3: Perbandingan *validation loss* pra-pelatihan seluruh *encoder*.

4.5.2 Analisis Kinerja Hasil *Joint Training*

Kinerja empat model utama dari konfigurasi JST berbasis xLSTM dan LSTM sebagai pembanding disajikan pada tabel 4.11 - 4.16. Evaluasi dilakukan menggunakan metrik yang telah dijelaskan pada subbab 2.9.

Tabel 4.11: Hasil evaluasi model pada dataset IMDB dengan $K = 30$

Model	Accuracy	F1 Macro	Precision	Recall	TC-W2V	TD
JST xLSTM[1:0]	0.8225 $\pm$ 0.0032	0.8225 $\pm$ 0.0032	0.8226 $\pm$ 0.0031	0.8225 $\pm$ 0.0032	0.5460 $\pm$ 0.0071	0.2989 $\pm$ 0.0087
JST xLSTM[0:1]	0.8288 $\pm$ 0.0007	0.8287 $\pm$ 0.0007	0.8292 $\pm$ 0.0011	0.8288 $\pm$ 0.0007	0.5588 $\pm$ 0.0062	0.2800 $\pm$ 0.0262
JST xLSTM[1:1]	0.8261 $\pm$ 0.0024	0.8261 $\pm$ 0.0024	0.8262 $\pm$ 0.0023	0.8261 $\pm$ 0.0024	0.5481 $\pm$ 0.0154	0.3144 $\pm$ 0.0231
JST xLSTM[7:1]	0.8277 $\pm$ 0.0029	0.8276 $\pm$ 0.0031	0.8290 $\pm$ 0.0018	0.8277 $\pm$ 0.0029	0.5436 $\pm$ 0.0141	0.3567 $\pm$ 0.0425
LSTM	0.8112 $\pm$ 0.0041	0.8109 $\pm$ 0.0043	0.8118 $\pm$ 0.0038	0.8112 $\pm$ 0.0041	0.5602 $\pm$ 0.0093	0.2734 $\pm$ 0.0112

Tabel 4.12: Hasil evaluasi model pada dataset Yelp dengan $K = 30$

Model	Accuracy	F1 Macro	Precision	Recall	TC-W2V	TD
JST xLSTM[1:0]	0.8921 $\pm$ 0.0006	0.8921 $\pm$ 0.0006	0.8923 $\pm$ 0.0004	0.8921 $\pm$ 0.0006	0.4598 $\pm$ 0.0055	0.2800 $\pm$ 0.0178

Model	Accuracy	F1 Macro	Precision	Recall	TC-W2V	TD
JST xLSTM[0:1]	0.8876 $\pm$ 0.0022	0.8875 $\pm$ 0.0022	0.8884 $\pm$ 0.0028	0.8876 $\pm$ 0.0022	0.4452 $\pm$ 0.0057	0.2900 $\pm$ 0.0163
JST xLSTM[1:1]	0.8896 $\pm$ 0.0006	0.8896 $\pm$ 0.0006	0.8897 $\pm$ 0.0007	0.8896 $\pm$ 0.0006	0.4451 $\pm$ 0.0023	0.2844 $\pm$ 0.0150
JST xLSTM[7:1]	0.8927 $\pm$ 0.0024	0.8927 $\pm$ 0.0024	0.8930 $\pm$ 0.0025	0.8927 $\pm$ 0.0024	0.4598 $\pm$ 0.0041	0.3500 $\pm$ 0.0027
LSTM	0.8743 $\pm$ 0.0031	0.8742 $\pm$ 0.0032	0.8748 $\pm$ 0.0029	0.8743 $\pm$ 0.0031	0.4312 $\pm$ 0.0087	0.2621 $\pm$ 0.0143

Tabel 4.13: Hasil evaluasi model pada dataset IMDB dengan $K = 50$

Model	Accuracy	F1 Macro	Precision	Recall	TC-W2V	TD
JST xLSTM[1:0]	0.8225 $\pm$ 0.0034	0.8224 $\pm$ 0.0035	0.8227 $\pm$ 0.0032	0.8225 $\pm$ 0.0034	0.5364 $\pm$ 0.0047	0.2900 $\pm$ 0.0043
JST xLSTM[0:1]	0.8250 $\pm$ 0.0015	0.8249 $\pm$ 0.0015	0.8259 $\pm$ 0.0014	0.8250 $\pm$ 0.0015	0.5334 $\pm$ 0.0226	0.3033 $\pm$ 0.0544
JST xLSTM[1:1]	0.8243 $\pm$ 0.0028	0.8243 $\pm$ 0.0028	0.8245 $\pm$ 0.0028	0.8243 $\pm$ 0.0028	0.5453 $\pm$ 0.0022	0.2787 $\pm$ 0.0047
JST xLSTM[7:1]	0.8318 $\pm$ 0.0026	0.8318 $\pm$ 0.0026	0.8321 $\pm$ 0.0022	0.8318 $\pm$ 0.0026	0.5365 $\pm$ 0.0009	0.2993 $\pm$ 0.0152
LSTM	0.8098 $\pm$ 0.0052	0.8097 $\pm$ 0.0053	0.8103 $\pm$ 0.0049	0.8098 $\pm$ 0.0052	0.5112 $\pm$ 0.0078	0.3144 $\pm$ 0.0201

Tabel 4.14: Hasil evaluasi model pada dataset Yelp dengan $K = 50$

Model	Accuracy	F1 Macro	Precision	Recall	TC-W2V	TD
JST xLSTM[1:0]	0.8911 $\pm$ 0.0023	0.8911 $\pm$ 0.0023	0.8914 $\pm$ 0.0021	0.8911 $\pm$ 0.0023	0.4390 $\pm$ 0.0056	0.2658 $\pm$ 0.0117
JST xLSTM[0:1]	0.8871 $\pm$ 0.0062	0.8870 $\pm$ 0.0062	0.8874 $\pm$ 0.0059	0.8871 $\pm$ 0.0062	0.4391 $\pm$ 0.0043	0.2973 $\pm$ 0.0276
JST xLSTM[1:1]	0.8873 $\pm$ 0.0024	0.8873 $\pm$ 0.0024	0.8874 $\pm$ 0.0024	0.8873 $\pm$ 0.0024	0.4344 $\pm$ 0.0023	0.2767 $\pm$ 0.0066
JST xLSTM[7:1]	0.8960 $\pm$ 0.0006	0.8960 $\pm$ 0.0006	0.8965 $\pm$ 0.0007	0.8960 $\pm$ 0.0006	0.4272 $\pm$ 0.0078	0.2760 $\pm$ 0.0326
LSTM	0.8812 $\pm$ 0.0038	0.8811 $\pm$ 0.0039	0.8816 $\pm$ 0.0035	0.8812 $\pm$ 0.0038	0.4498 $\pm$ 0.0091	0.3012 $\pm$ 0.0158

Tabel 4.15: Hasil evaluasi model pada dataset IMDB dengan $K = 80$

Model	Accuracy	F1 Macro	Precision	Recall	TC-W2V	TD
JST xLSTM[1:0]	0.8251 $\pm$ 0.0026	0.8251 $\pm$ 0.0026	0.8252 $\pm$ 0.0025	0.8251 $\pm$ 0.0026	0.5120 $\pm$ 0.0098	0.2917 $\pm$ 0.0129
JST xLSTM[0:1]	0.8261 $\pm$ 0.0019	0.8261 $\pm$ 0.0019	0.8264 $\pm$ 0.0018	0.8261 $\pm$ 0.0019	0.5061 $\pm$ 0.0390	0.2908 $\pm$ 0.0583
JST xLSTM[1:1]	0.8242 $\pm$ 0.0012	0.8240 $\pm$ 0.0013	0.8255 $\pm$ 0.0008	0.8242 $\pm$ 0.0012	0.5278 $\pm$ 0.0032	0.2617 $\pm$ 0.0062

Model	Accuracy	F1 Macro	Precision	Recall	TC-W2V	TD
JST xLSTM[7:1]	0.8323 $\pm$ 0.0011	0.8322 $\pm$ 0.0011	0.8324 $\pm$ 0.0011	0.8323 $\pm$ 0.0011	0.4976 $\pm$ 0.0021	0.3438 $\pm$ 0.0226
LSTM	0.8089 $\pm$ 0.0047	0.8088 $\pm$ 0.0048	0.8094 $\pm$ 0.0044	0.8089 $\pm$ 0.0047	0.5312 $\pm$ 0.0115	0.2801 $\pm$ 0.0193

Tabel 4.16: Hasil evaluasi model pada dataset Yelp dengan $K = 80$

Model	Accuracy	F1 Macro	Precision	Recall	TC-W2V	TD
JST xLSTM[1:0]	0.8933 $\pm$ 0.0030	0.8933 $\pm$ 0.0030	0.8944 $\pm$ 0.0022	0.8933 $\pm$ 0.0030	0.4173 $\pm$ 0.0065	0.2624 $\pm$ 0.0119
JST xLSTM[0:1]	0.8913 $\pm$ 0.0014	0.8913 $\pm$ 0.0014	0.8915 $\pm$ 0.0015	0.8913 $\pm$ 0.0014	0.4236 $\pm$ 0.0165	0.2628 $\pm$ 0.0218
JST xLSTM[1:1]	0.8909 $\pm$ 0.0010	0.8909 $\pm$ 0.0010	0.8910 $\pm$ 0.0009	0.8909 $\pm$ 0.0010	0.4262 $\pm$ 0.0059	0.2758 $\pm$ 0.0159
JST xLSTM[7:1]	0.9001 $\pm$ 0.0002	0.9001 $\pm$ 0.0003	0.9003 $\pm$ 0.0001	0.9001 $\pm$ 0.0002	0.4156 $\pm$ 0.0047	0.3071 $\pm$ 0.0133
LSTM	0.8701 $\pm$ 0.0042	0.8700 $\pm$ 0.0043	0.8706 $\pm$ 0.0039	0.8701 $\pm$ 0.0042	0.4389 $\pm$ 0.0108	0.3124 $\pm$ 0.0176

4.5.2.1 Analisis Kinerja Hasil Analisis Sentimen

Hasil evaluasi kinerja analisis sentimen pada seluruh kombinasi model, *dataset*, dan jumlah topik disajikan pada Tabel 4.11 hingga Tabel 4.16. Secara umum, seluruh konfigurasi xLSTM menunjukkan performa yang stabil pada kedua *dataset*, dengan nilai *accuracy* dan F1 *macro* yang relatif tidak berfluktuasi signifikan terhadap perubahan jumlah topik K .

Pada *dataset* IMDB, xLSTM[7:1] mencapai performa terbaik pada $K = 50$ dan $K = 80$ dengan *accuracy* masing-masing sebesar 0.8318 dan 0.8323. Sementara pada $K = 30$, xLSTM[0:1] sedikit lebih unggul dengan *accuracy* sebesar 0.8288, menunjukkan bahwa konfigurasi dengan satu blok sLSTM mampu bersaing pada jumlah topik yang lebih kecil. Secara keseluruhan, perbedaan performa antar konfigurasi xLSTM pada *dataset* IMDB relatif kecil.

Pada *dataset* Yelp, xLSTM[7:1] secara konsisten mengungguli seluruh konfigurasi lainnya pada semua variasi K , dengan nilai *accuracy* tertinggi sebesar 0.9001 pada $K = 80$. Performa pada *dataset* Yelp secara keseluruhan lebih tinggi dibandingkan IMDB. Konfigurasi xLSTM[1:0] dan xLSTM[1:1] menunjukkan performa yang sebanding, sementara xLSTM[0:1] secara konsisten mencatat nilai terendah di antara seluruh konfigurasi xLSTM pada kedua *dataset*.

Encoder LSTM sebagai pembanding menunjukkan performa yang lebih rendah dibandingkan seluruh konfigurasi xLSTM pada semua skenario pengujian, yang

mengindikasikan bahwa mekanisme memori yang lebih canggih pada xLSTM memberikan keunggulan dalam membentuk representasi dokumen yang lebih diskriminatif untuk tugas klasifikasi sentimen.

4.5.2.2 Analisis Kinerja Hasil Pendeteksian Topik

4.6 Analisis Interpretabilitas Model

BAB 5

PENUTUP

5.1 Kesimpulan

5.2 Saran

DAFTAR PUSTAKA

- Aggarwal, C. C. et al. (2018). *Neural networks and deep learning*, volume 10. Springer.
- Alharthi, M. & Mahmood, A. (2024). xlstmtime: Long-term time series forecasting with xlstm. *AI*, 5(3), 1482–1495. <https://doi.org/10.3390/ai5030071>
- Almeida, F. & Xexéo, G. (2023). *Word embeddings: A survey*. <https://arxiv.org/abs/1901.09069>
- Andreotta, M., Nugroho, R., Hurlstone, M., Boschetti, F., Farrell, S., Walker, I., & Paris, C. (2019). Analyzing social media data: A mixed-methods framework combining computational and qualitative text analysis. *Behavior Research Methods*, 51. <https://doi.org/10.3758/s13428-019-01202-8>
- Anton, H. (2013). *Elementary Linear Algebra*. Wiley. <https://books.google.co.id/books?id=neYGCwAAQBAJ>
- Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). *Layer normalization*. <https://arxiv.org/abs/1607.06450>
- Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J., & Hochreiter, S. (2024). *xlstm: Extended long short-term memory*. <https://arxiv.org/abs/2405.04517>
- Behdenna, S., Barigou, F., & Belalem, G. (2018). Document level sentiment analysis: A survey. *EAI Endorsed Transactions on Context-aware Systems and Applications*, 4(13). <https://doi.org/10.4108/eai.14-3-2018.154339>
- Bengio, Y., Courville, A. C., & Vincent, P. (2012). Unsupervised feature learning and deep learning: A review and new perspectives. *CoRR*, abs/1206.5538. <http://arxiv.org/abs/1206.5538>
- Bishop, C. M. & Nasrabadi, N. M. (2006). *Pattern recognition and machine learning*, volume 4. Springer.
- Blei, D. M. (2012). Probabilistic topic models. *Commun. ACM*, 55(4), 77–84. <https://doi.org/10.1145/2133806.2133826>
- Choi, Y. & Cardie, C. (2009). Adapting a polarity lexicon using integer linear programming for domain-specific sentiment classification. <https://doi.org/10.3115/1699571.1699590>

- Da Silva, I. N., Hernane Spatti, D., Andrade Flauzino, R., Liboni, L. H. B., & dos Reis Alves, S. F. (2016). Artificial neural network architectures and training processes. *Artificial Neural Networks: A Practical Course*, 21–28. Springer.
- Dieng, A. B., Ruiz, F. J. R., & Blei, D. M. (2019). Topic modeling in embedding spaces. *CoRR*, abs/1907.04907. <http://arxiv.org/abs/1907.04907>
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). *An image is worth 16x16 words: Transformers for image recognition at scale*. <https://arxiv.org/abs/2010.11929>
- Dubey, S. R., Singh, S. K., & Chaudhuri, B. B. (2022). *Activation functions in deep learning: A comprehensive survey and benchmark*. <https://arxiv.org/abs/2109.14545>
- Elharrouss, O., Mahmood, Y., Bechqito, Y., Serhani, M. A., Badidi, E., Riffi, J., & Tairi, H. (2025). *Task-based loss functions in computer vision: A comprehensive review*. <https://arxiv.org/abs/2504.04242>
- Fu, X., Wu, H., & Cui, L. (2016). Topic sentiment joint model with word embeddings. *Proceedings of the DMNLP Workshop at ECML/PKDD*. <https://ceur-ws.org/Vol-1646/paper6.pdf>. Extended version available as TSWE: Topic Sentiment Joint Model with Word Embeddings
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. The MIT Press.
- Gowandi, T., Murfi, H., & Nurrohmah, S. (2021). Performance analysis of hybrid architectures of deep learning for indonesian sentiment analysis. *Soft Computing in Data Science - 6th International Conference, SCDS 2021, Proceedings, Communications in Computer and Information Science*, 18–27. https://doi.org/10.1007/978-981-16-7334-4_2. Publisher Copyright: © 2021, Springer Nature Singapore Pte Ltd.; 6th International Conference on Soft Computing in Data Science, SCDS 2021 ; Conference date: 02-11-2021 Through 03-11-2021
- Gui, L., Leng, J., Zhou, J., Xu, R., & He, Y. (2022). Multi task mutual learning for joint sentiment classification and topic detection. *IEEE Transactions on Knowledge and Data Engineering*, 34(4), 1915–1927. <https://doi.org/10.1109/TKDE.2020.2999489>
- Guo, W., Lin, K., & Ye, W. (2021). *Deep embedded k-means clustering*. <https://arxiv.org/abs/2109.15149>

- Hendrycks, D. & Gimpel, K. (2023). *Gaussian error linear units (gelus)*. <https://arxiv.org/abs/1606.08415>
- Ho, W. K., Ho, F., & Lee, T. (2012). Exponential function and its derivative revisited. *International Journal of Mathematical Education in Science and Technology*, 44, 1–6. <https://doi.org/10.1080/0020739X.2012.703341>
- Huang, L., Qin, J., Zhou, Y., Zhu, F., Liu, L., & Shao, L. (2020). Normalization techniques in training dnns: Methodology, analysis and application. *CoRR*, abs/2009.12836. <https://arxiv.org/abs/2009.12836>
- Jain, A. K., Murty, M. N., & Flynn, P. J. (1999). Data clustering: a review. *ACM Comput. Surv.*, 31(3), 264–323. <https://doi.org/10.1145/331499.331504>
- Keogh, E. & Mueen, A. (2010). *Curse of Dimensionality*, 257–258. Springer US. https://doi.org/10.1007/978-0-387-30164-8_192
- Kingma, D. P. & Ba, J. (2017). *Adam: A method for stochastic optimization*. <https://arxiv.org/abs/1412.6980>
- Lin, C. (2009). Joint sentiment/topic model for sentiment analysis. *International Conference on Information and Knowledge Management, Proceedings*. <https://doi.org/10.1145/1645953.1646003>
- Liu, B. (2012). Sentiment analysis and opinion mining. *Sentiment Analysis and Opinion Mining*, volume 5. <https://doi.org/10.2200/S00416ED1V01Y201204HLT016>
- Loshchilov, I. & Hutter, F. (2017). Fixing weight decay regularization in adam. *CoRR*, abs/1711.05101. <http://arxiv.org/abs/1711.05101>
- Luong, M.-T., Pham, H., & Manning, C. D. (2015). *Effective approaches to attention-based neural machine translation*. <https://arxiv.org/abs/1508.04025>
- Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning word vectors for sentiment analysis. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, 142–150. <https://aclanthology.org/P11-1015/>
- Mienye, I. D., Swart, T. G., & Obaido, G. (2024). Recurrent neural networks: A comprehensive review of architectures, variants, and applications. *Information*, 15(9). <https://doi.org/10.3390/info15090517>

- Nguyen, T. H. & Shirai, K. (2015). Topic modeling based sentiment analysis on social media for stock market prediction. *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 1354–1364. <https://doi.org/10.3115/v1/P15-1131>
- Nie, Y., Nguyen, N. H., Sinthong, P., & Kalagnanam, J. (2023). *A time series is worth 64 words: Long-term forecasting with transformers*. <https://arxiv.org/abs/2211.14730>
- Nieradzik, L., Scheuermann, G., Saur, D., & Gillmann, C. (2021). *Effect of the output activation function on the probabilities and errors in medical image segmentation*. <https://arxiv.org/abs/2109.00903>
- Nikhil, Nikhil Tikoo, S. K. H. S. P. D. P. G. R. (2015). A survey on text mining and sentiment analysis for unstructured web data. *SSRN Electronic Journal*, 02, 1292–1297.
- O’Callaghan, D., Greene, D., Carthy, J., & Cunningham, P. (2015). An analysis of the coherence of descriptors in topic modeling. *Expert Systems with Applications*, 42(13), 5645–5657. <https://doi.org/https://doi.org/10.1016/j.eswa.2015.02.055>
- Pang, B. & Lee, L. (2008). Opinion mining and sentiment analysis. *Found. Trends Inf. Retr.*, 2(1–2), 1–135. <https://doi.org/10.1561/15000000011>
- Petrus, J., Ermatita, & Sukemi (2019). Soft and hard clustering for abstract scientific paper in indonesian. *2019 International Conference on Informatics, Multimedia, Cyber and Information System (ICIMCIS)*, 131–136. <https://doi.org/10.1109/ICIMCIS48181.2019.8985231>
- Pradipta Hardinatha, E., Malikulmulki Arasy, J., Walidain, C., Multazam, I. H., Athallah Adnan, R., & Aulia Adnan, R. (2025). *Large language models via rust: The state of the art open foundational models in rust*. <https://lmvr.rantai.dev/docs/lmvr/>. Accessed: 2026-01-11.
- Qamar, R. & Zardari, B. (2023). Artificial neural networks: An overview. *Mesopotamian Journal of Computer Science*, 2023, 130–139. <https://doi.org/10.58496/MJCSC/2023/015>
- Radford, A. & Narasimhan, K. (2018). Improving language understanding by generative pre-training. <https://api.semanticscholar.org/CorpusID:49313245>

- Ramachandran, P., Zoph, B., & Le, Q. (2017). Swish: a self-gated activation function. <https://doi.org/10.48550/arXiv.1710.05941>
- Röder, M., Both, A., & Hinneburg, A. (2015). Exploring the space of topic coherence measures. *Proceedings of the Eighth ACM International Conference on Web Search and Data Mining, WSDM 2015, Shanghai, China, February 2-6, 2015*, 399–408. <https://doi.org/10.1145/2684822.2685324>
- Ruder, S. (2017). *An overview of multi-task learning in deep neural networks*. <https://arxiv.org/abs/1706.05098>
- Schmidt, R. M. (2019). *Recurrent neural networks (rnns): A gentle introduction and overview*. <https://arxiv.org/abs/1912.05911>
- Sengupta, A., Paka, W. S., Roy, S., Ranjan, G., & Chakraborty, T. (2021). *An embedding-based joint sentiment-topic model for short texts*. <https://arxiv.org/abs/2103.14410>
- Sokolova, M. & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing Management*, 45(4), 427–437. <https://doi.org/10.1016/j.ipm.2009.03.002>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023). *Attention is all you need*. <https://arxiv.org/abs/1706.03762>
- Vennerød, C. B., Kjærran, A., & Bugge, E. S. (2021). *Long short-term memory rnn*. <https://arxiv.org/abs/2105.06756>
- Wu, Y. & He, K. (2018). *Group normalization*. <https://arxiv.org/abs/1803.08494>
- Xu, G., Wang, X., Wu, X., Leng, X., & Xu, Y. (2025). Development of residual learning in deep neural networks for computer vision: A survey. *Engineering Applications of Artificial Intelligence*, 142, 109890. <https://doi.org/10.1016/j.engappai.2024.109890>
- Zafarani-Moattar, E., Kangavari, M. R., & Rahmani, A. M. (2024). *A comprehensive study on frequent pattern mining and clustering categories for topic detection in persian text stream*. <https://arxiv.org/abs/2403.10237>
- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into deep learning*. <https://arxiv.org/abs/2106.11342>

- Zhang, L., Wang, S., & Liu, B. (2018). *Deep learning for sentiment analysis : A survey*. <https://arxiv.org/abs/1801.07883>
- Zhang, W., Li, X., Deng, Y., Bing, L., & Lam, W. (2022). *A survey on aspect-based sentiment analysis: Tasks, methods, and challenges*. <https://arxiv.org/abs/2203.01054>
- Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level convolutional networks for text classification. *arXiv:1509.01626 [cs]*. https://huggingface.co/datasets/fancyzhx/yelp_polarity

Lampiran 1 *Code* Pra-pelatihan xLSTM

**Lampiran 2 *Code Joint Sentiment–Topic* berbasis xLSTM dengan Pendekatan
*Multi–Task Learning***

